

Physics-Informed Neural Networks (PINNs) for a 1D Heat Diffusion Problem

Quentin Pontalier

September 23, 2026

Contents

1	Introduction	2
1.1	Artificial Intelligence and Scientific Research	2
1.2	Scientific Machine Learning	2
1.3	Physics-Informed Neural Networks (PINNs)	3
1.4	Objectives of the Present Work	4
2	PINN Architecture and Training Strategy	5
2.1	General Neural Network Architecture	5
2.2	Activation Functions	9
2.3	Activation Functions and the Need for Non-Linearity	9
2.4	Automatic Differentiation	11
2.5	Weight Initialization Strategy	11
2.5.1	Glorot Uniform Initialization	12
2.5.2	Importance for PINNs	12
2.5.3	Alternative Initialization Methods	13
2.5.4	Initialization and Optimization Dynamics	14
2.6	Collocation Points	14
2.7	Collocation Points and Sampling Strategies	14
2.8	Boundary and Initial Condition Enforcement	15
2.8.1	Soft Constraint Formulation	16
2.8.2	Hard Constraint Formulation	17
2.8.3	Comparison Between Soft and Hard Constraints	18
2.9	Optimization Strategy	18
2.10	Floating Point Precision	18
2.11	Random Seeds and Reproducibility	18
2.11.1	Weight Initialization Seed	19
2.11.2	Collocation Point Seed	19
2.11.3	Dataset Generation Seed	20
2.11.4	Training and Optimization Randomness	20
2.11.5	Importance of Multi-Seed Robustness Studies	21
2.11.6	Seed Management in Practice	21
2.12	Role of the Forward PINN Benchmark	22
3	PINN Implementations and Computational Frameworks	22
3.1	TensorFlow-Based Implementations	22
3.2	PyTorch-Based Implementations	23
3.3	JAX-Based Implementations	24
3.4	Comparison Between Frameworks	25

3.5	General Challenges in PINN Implementations	25
4	One-Dimensional Heat Diffusion Problem	26
4.1	Governing Equation	26
4.2	Initial and Boundary Conditions	26
4.3	Derivation of the Exact Solution	27
4.3.1	Temporal Equation	27
4.3.2	Spatial Equation	27
4.4	Particular Solution Used in This Work	28
5	Forward Problem	29
5.1	Normalization Strategies	30
5.1.1	Fully Non-Dimensionalized Formulation	30
5.1.2	Partially Normalized Formulation	31
5.1.3	Non-Normalized Physical Formulation	31
5.1.4	Influence of Normalization on Inverse Problems	32
6	Inverse Problem	32
6.1	Why PINNs Are Particularly Interesting for Inverse Problems	33
6.2	Inverse Problem Formulation	33
6.3	Soft and Hard Constraint Strategies	34
6.4	Enforcement of Initial and Boundary Conditions	34
6.5	Low-Amplitude Solutions and Parameter Identifiability	35
6.5.1	Parameter Reparameterization	36
6.5.2	Sensitivity-Based Weighting	38
6.6	Noise Sensitivity and Identifiability Challenges	42
6.7	Negative Temperature Measurements	49
6.8	Adaptive Weighting Strategies	51
6.9	Galerkin-Inspired Constraints	52
6.10	Dynamic MSE Thresholds and Robustness Studies	52
6.11	Limitations and Perspectives	53

Disclaimer

This document was written as part of a personal research project on Physics-Informed Neural Networks (PINNs). It is primarily intended for scientists and engineers with a background in physics, mechanical engineering, applied mathematics, or numerical simulation who are interested in transitioning toward Scientific Machine Learning (SciML) and PINNs.

Readers unfamiliar with machine learning may benefit from completing an introductory machine learning course before studying some of the more advanced concepts discussed herein. The author personally completed the [Machine Learning Specialization](#) and [Deep Learning Specialization](#) offered by DeepLearning.AI and taught by Andrew Ng. While these courses are not specifically focused on SciML or PINNs they provide a broad introduction to the fundamental concepts of modern machine learning and offer a sufficient foundation for readers with a scientific or engineering background to approach the methodologies presented in this document.

Large Language Models (LLMs) were occasionally used to assist with writing, proofreading, and structuring this paper. However, all technical discussions, implementations, analyses, interpretations, and conclusions were reviewed and validated by the author. Unless otherwise stated, all figures presented in this report were generated by the author.

1 Introduction

1.1 Artificial Intelligence and Scientific Research

Over the last decade, Artificial Intelligence (AI) has profoundly transformed modern society. Advances in machine learning, deep learning, and more recently Large Language Models (LLMs) have led to major breakthroughs in numerous domains, including natural language processing (the automatic understanding and generation of human language), computer vision (the extraction of information from images and videos), healthcare, finance, autonomous systems, robotics, and industrial optimization. These technologies are increasingly capable of automating complex tasks, extracting information from massive datasets, and assisting human decision-making processes. While many early applications focused on classification, prediction, and pattern recognition, AI is now increasingly being used to support scientific research itself.

Scientific disciplines have traditionally relied on a combination of theoretical modeling, experimental measurements, and numerical simulations. Compared with many machine learning application domains, scientific applications often impose additional requirements such as physical consistency, interpretability, reproducibility, and rigorous validation. This naturally creates a certain degree of inertia, as new computational methodologies must undergo extensive testing before gaining widespread acceptance. Despite these challenges, researchers have increasingly embraced machine learning techniques as powerful tools for accelerating discovery, improving data analysis, enhancing predictive modeling capabilities, and complementing traditional numerical simulation methods.

1.2 Scientific Machine Learning

The convergence of machine learning and scientific computing has given rise to a rapidly growing field known as Scientific Machine Learning (SciML). The objective of SciML is to combine data-driven learning with physics-based modeling. Machine learning provides flexible tools for learning complex relationships directly from data, whereas physical models incorporate established scientific knowledge about the system under study. By leveraging both sources of information, SciML seeks to improve predictive accuracy, reduce data requirements, and ensure greater physical consistency of the resulting models. Several important families of SciML methods have recently emerged, including:

- Physics-Informed Neural Networks (PINNs),

- Deep Operator Networks (DeepONets),
- Neural Operators,
- Fourier Neural Operators (FNOs),
- Graph Neural Networks (GNNs),
- and hybrid physics-data models.

Deep Operator Networks (DeepONets) and Neural Operators can be viewed as machine learning methods that attempt to learn how to solve a family of physical problems rather than a single instance of a problem. Their objective is to build models capable of rapidly predicting approximate solutions once they have been trained on a sufficiently large number of examples. Fourier Neural Operators (FNOs) are a particular type of Neural Operator that have recently attracted significant attention due to their ability to efficiently approximate solutions of certain classes of partial differential equations. Graph Neural Networks (GNNs) are designed to work with systems that can naturally be represented as interconnected objects. Such representations are frequently encountered in engineering applications involving complex geometries, computational meshes, molecular structures, or networked systems.

Although these approaches differ considerably in their implementation and underlying mathematical foundations, they share a common goal: integrating machine learning with scientific modeling to enhance predictive capabilities, accelerate numerical simulations, and extract meaningful information from physical systems. Current applications of SciML in physics include, but are not limited to:

- fluid mechanics (e.g., turbulence modeling and flow reconstruction),
- heat transfer (e.g., thermal property estimation and surrogate modeling),
- solid mechanics (e.g., stress and deformation prediction),
- electromagnetics (e.g., wave propagation and antenna design),
- climate science (e.g., reduced-order weather forecasting),
- geophysics (e.g., subsurface property identification),
- materials science (e.g., microstructure-property relationships),
- plasma physics (e.g., fusion reactor modeling),
- biomedical engineering (e.g., blood flow and medical imaging),
- and quantum systems (e.g., Schrödinger equation solvers and quantum control).

1.3 Physics-Informed Neural Networks (PINNs)

Among the various SciML methodologies, Physics-Informed Neural Networks (PINNs) have emerged as one of the most actively studied approaches since the seminal work of Raissi et al (*Journal of Computational Physics*, 2019). The key idea behind PINNs is relatively simple: instead of learning solely from data, the neural network is constrained to satisfy the governing physical equations during training.

In a traditional supervised machine learning problem, a neural network is provided with examples of known inputs and outputs. The objective is to progressively adjust the network parameters so that its predictions become as close as possible to the observed outputs. For example, a neural network may be trained to predict the temperature at different positions

and times within a metal bar using measurements collected during a heat diffusion experiment. By comparing its predictions to the measured temperatures, the network gradually learns the relationship between the inputs (position and time) and the desired output (temperature). To quantify the discrepancy between predictions and observations, a numerical quantity called the *loss function* is introduced. The loss can be viewed as a measure of prediction error that the optimization algorithm attempts to minimize during training. The larger the prediction error, the larger the loss.

PINNs extend this idea by incorporating physical knowledge directly into the loss function. In addition to matching the available data, the network is also penalized whenever its predictions violate the governing differential equations. This is made possible through *automatic differentiation*, a computational technique that allows derivatives of the neural network output to be computed efficiently and accurately. These derivatives are then used to evaluate how well the network satisfies the underlying physical equations. In the heat diffusion example, the network is therefore encouraged not only to reproduce the measured temperatures, but also to satisfy the heat equation throughout the domain. As a result, PINNs generate solutions that are consistent not only with the available data but also with the governing physics.

This approach is particularly attractive for engineering applications because many practical problems involve both physical knowledge and limited experimental data. In such situations, neither purely data-driven methods nor purely physics-based approaches may be sufficient on their own. Data-driven methods often require large amounts of high-quality data to achieve accurate predictions, which may be difficult or expensive to obtain in many scientific and engineering applications. Conversely, purely physics-based models rely on accurate governing equations, boundary conditions, and material parameters, which are not always fully known in practice. By combining observational data with physical knowledge, PINNs aim to leverage the strengths of both approaches while mitigating their respective limitations.

Since their introduction, PINNs have been successfully applied to a wide variety of problems, including heat diffusion, fluid mechanics, wave propagation, elasticity, electromagnetics, inverse parameter identification, uncertainty quantification, data assimilation, multiphysics systems, and more recently quantum systems and quantum control problems. One of the most promising applications of PINNs concerns inverse problems. In many engineering situations, physical parameters cannot be measured directly and must instead be inferred from sparse and potentially noisy observations. Examples include estimating thermal diffusivities, material properties, permeability coefficients, reaction rates, or boundary conditions from experimental measurements. By simultaneously incorporating observational data and physical constraints within a unified optimization framework, PINNs offer an attractive alternative to traditional parameter estimation techniques.

1.4 Objectives of the Present Work

The present work focuses on PINNs applied to a simple one-dimensional heat diffusion problem. Although the one-dimensional heat equation has already been extensively studied in the PINN literature, it remains one of the few benchmark problems for which every aspect of the learning process can be investigated in a fully controlled environment. The availability of an analytical solution makes it possible to isolate and study individual mechanisms such as optimization dynamics, parameter identifiability, loss balancing, convergence behavior, and robustness to noisy data without the additional complexity introduced by more realistic applications. Furthermore, many of the difficulties encountered in large-scale PINN applications already emerge in this simplified setting, including optimization stiffness, parameter non-identifiability, sensitivity to noise, collocation point selection (points in the computational domain where the governing equations are enforced), and convergence issues.

The study considers both the forward and inverse formulations of the problem. In the forward problem, the thermal diffusivity coefficient is assumed to be known, and the objective is

to reconstruct the temperature field satisfying the governing heat equation together with the prescribed initial and boundary conditions. In the inverse problem, the objective is to estimate the thermal diffusivity coefficient α directly from noisy temperature measurements while simultaneously reconstructing the temperature field. This task is representative of many real-world scientific and engineering applications in which physical parameters cannot be measured directly and must instead be inferred from experimental observations.

Particular attention is devoted to understanding why PINNs may succeed or fail even on apparently simple benchmark problems. Rather than viewing the one-dimensional heat equation as a trivial example, this work treats it as a controlled laboratory for studying parameter identifiability, optimization dynamics, noise robustness, loss balancing strategies, sensitivity-based weighting approaches, hard versus soft constraint formulations, and Galerkin-inspired techniques designed to improve inverse parameter reconstruction.

The remainder of this document is organized as follows. Section 2 presents the PINN architecture, training strategies, initialization procedures, normalization techniques, and reproducibility considerations. Section 3 introduces the different machine learning frameworks investigated throughout this work. Section 4 describes the one-dimensional heat diffusion problem and derives its analytical solution. Section 5 investigates the forward PINN formulation, while Section 6 focuses on the inverse problem and the various strategies developed to improve parameter identification. Finally, the results and conclusions are presented together with perspectives for future developments.

Beyond the one-dimensional benchmark considered here, the long-term objective is to develop the methodological foundations required to tackle more realistic scientific and engineering problems for which analytical solutions are unavailable, such as systems governed by nonlinear partial differential equations or involving complex geometries. In such situations, validation must rely on experimental measurements, high-fidelity numerical simulations, or both. For the author, whose background lies in engineering physics, experimental research, and numerical simulation, this project also represents a transition toward the field of Scientific Machine Learning and a broader effort to understand how machine learning techniques can complement established scientific methodologies.

2 PINN Architecture and Training Strategy

This section presents the general architecture and training methodology used for the Physics-Informed Neural Networks (PINNs) developed in this work. The objective is to construct a neural network capable of approximating the solution of the governing physical problem while simultaneously satisfying the underlying partial differential equation (PDE).

Unlike traditional supervised learning approaches that rely exclusively on observational data, PINNs integrate physical laws directly into the optimization process through automatic differentiation and physics-based loss functions. The neural network therefore acts as a continuous function approximator constrained by both available data and fundamental physical principles.

2.1 General Neural Network Architecture

The temperature field is approximated by a fully connected feedforward neural network,

$$T_{\theta}(x, t) = \mathcal{N}_{\theta}(x, t), \tag{1}$$

where

- $\mathcal{N}_{\theta}$ denotes the neural network,
- θ represents the set of trainable parameters, including all weights and biases,

- (x, t) are the spatial and temporal coordinates provided as inputs.

The objective of the neural network is to learn the unknown mapping between the spatial-temporal coordinates (x, t) and the corresponding temperature field $T(x, t)$. The network takes the coordinates (x, t) as inputs and returns the predicted temperature value $T_\theta(x, t)$. The NN architecture for the 1D thermal problem is composed of

- an input layer containing the spatial and temporal coordinates,
- several fully connected hidden layers, each composed of multiple neurons,
- an output layer producing the predicted temperature field.

At the most fundamental level, a neural network is composed of interconnected computational units called neurons. Each neuron receives information from the previous layer, computes a weighted sum of its inputs, adds a bias term, and applies a nonlinear activation function. Mathematically, the output of a neuron can be expressed as

$$a = \sigma \left(\sum_i w_i x_i + b \right), \quad (2)$$

where x_i are the inputs, w_i are the trainable weights, b is a bias term, and $\sigma(\cdot)$ denotes the activation function.

The role of a neuron can be viewed as transforming information received from the previous layer into a new representation. In the first hidden layers, neurons often learn relatively simple features of the target function, while deeper layers combine these elementary features into increasingly complex representations. Through the collective action of many neurons, the network progressively constructs an approximation of the solution $T_\theta(x, t)$.

For the one-dimensional heat diffusion problem considered in this work, a neuron may, for example, learn to respond preferentially to a specific region of the spatial domain or to a particular stage of the temporal evolution. Some neurons may contribute to describing the temperature gradients near the boundaries, while others may capture the global decay of temperature over time. The final temperature field is obtained by combining the contributions of all neurons through the network layers. Although such interpretations can be useful for small networks, the role of individual neurons becomes increasingly difficult to identify as the network size grows, since the learned representation is distributed across many neurons.

The specialization of neurons is not prescribed explicitly. At the beginning of training, the network parameters are initialized (see section). As the optimization progresses, the weights and biases are adjusted through backpropagation in order to minimize the loss function. Neurons therefore gradually adapt to the features that are most useful for reducing the prediction error and satisfying the physical constraints. While the initial random weights influence the early stages of training and may lead to slightly different learned representations, the final organization of the neurons is primarily driven by the optimization process rather than by the initialization itself. Consequently, two networks trained from different random initializations may use different internal neuron representations while still converging toward nearly identical temperature fields.

In a fully connected architecture, every neuron of a given layer is connected to every neuron of the following layer through trainable weights. During training, these parameters are continuously adjusted through *backpropagation* so that the network progressively improves its approximation of the target solution. The weights determine how strongly each input influences the response of a neuron. Large positive weights increase the importance of a particular input, whereas small or negative weights reduce its contribution. During training, the network automatically adjusts these weights to identify the combinations of spatial and temporal coordinates that best describe the temperature field. Biases play a complementary role. While the weights control the

slope and orientation of the neuron response, the bias shifts this response and determines the activation threshold of the neuron. Without biases, all neuron responses would be constrained to pass through the origin, significantly limiting the family of functions that the network can represent.

As a simple illustration, consider a neuron computing

$$a = \sigma(wx + b). \quad (3)$$

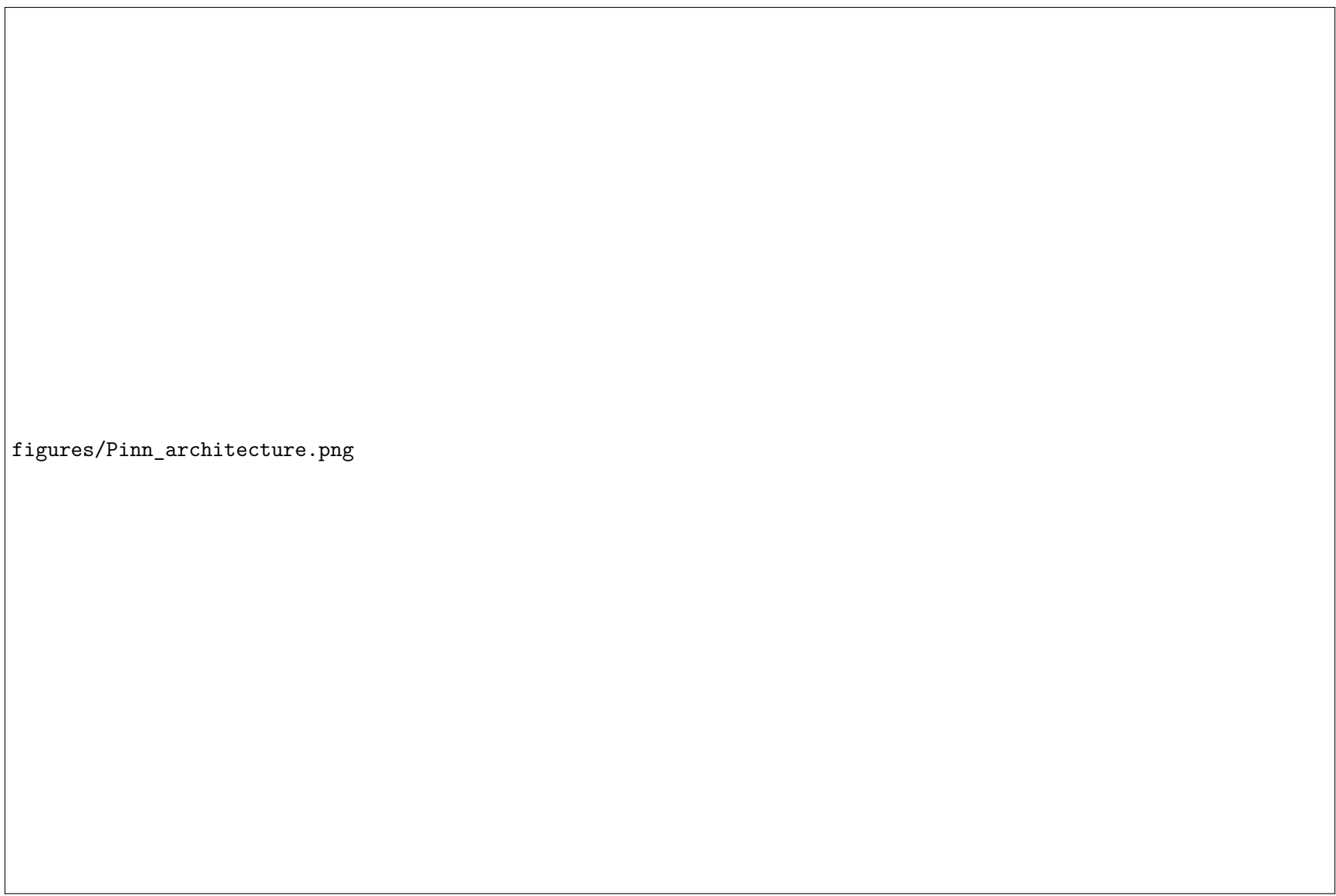
The weight w controls how rapidly the neuron reacts to variations of the input x , whereas the bias b determines the location where the neuron becomes active. Together, weights and biases provide the flexibility required to approximate complex nonlinear functions accurately.

For the one-dimensional heat diffusion problem considered here, the weights may adapt to capture spatial or temporal variations of the temperature field, while the biases allow the corresponding features to appear at different locations in space and time. Although the influence of an individual weight or bias is generally difficult to interpret, their collective adjustment enables the network to approximate the solution of the governing physical problem.

Although the mathematical operation performed by an individual neuron is relatively simple, the behavior of a deep neural network containing hundreds or thousands of neurons is generally difficult to interpret. While some neurons may appear to specialize in particular features, there is usually no straightforward physical meaning associated with a specific neuron. Instead, the learned representation is distributed across many neurons and layers. For this reason, neural networks are often considered as *black-box* models, although considerable research efforts are currently devoted to improving their interpretability.

The number of hidden layers (depth) and the number of neurons per layer (width) are important hyperparameters that can significantly influence both the approximation capability and optimization behavior of the PINN. Increasing the width generally increases the number of features that can be represented within a layer, whereas increasing the depth enables the network to construct more complex hierarchical representations of the solution. Unlike conventional neural networks, PINNs also incorporate physical constraints through the computation of PDE residuals using automatic differentiation. These residuals are included in the loss function and guide the optimization process toward solutions that satisfy the governing physical equations.

A general illustration of the PINN architecture used throughout this work is presented in Fig. 1.



figures/Pinn_architecture.png

Figure 1: General architecture of a Physics-Informed Neural Network (PINN) applied to the one-dimensional heat equation. The network receives the spatial and temporal coordinates as inputs and predicts the temperature field. Automatic differentiation is then used to compute the PDE residual and enforce the physical constraints during training.

2.2 Activation Functions

2.3 Activation Functions and the Need for Non-Linearity

In a neural network, each neuron first computes a weighted sum of its inputs,

$$z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}, \quad (4)$$

where $W^{(\ell)}$ and $b^{(\ell)}$ denote the weights and biases of layer ℓ , respectively. The result is then passed through an activation function $\sigma(\cdot)$,

$$a^{(\ell)} = \sigma\left(z^{(\ell)}\right). \quad (5)$$

The activation function is a key component of neural networks because it introduces non-linearity into the model.

Without an activation function, every layer would simply perform a linear transformation of its inputs. Since the composition of multiple linear transformations remains linear, a deep neural network composed only of linear operations would be mathematically equivalent to a single linear layer, regardless of the number of hidden layers. In such a case, increasing the depth of the network would provide no additional learning capability.

As a simple analogy, stacking several straight rulers on top of each other still produces a straight line. To represent complex patterns, curves, and nonlinear relationships, the network must include nonlinear operations between layers.

Nonlinear activation functions allow neural networks to approximate highly complex functions and are therefore essential for solving realistic engineering and scientific problems. In fact, the Universal Approximation Theorem states that a sufficiently large neural network equipped with nonlinear activation functions can approximate a wide range of continuous functions with arbitrary accuracy.

In classical supervised machine learning, one of the most widely used activation functions is the Rectified Linear Unit (ReLU), defined as

$$\sigma_{\text{ReLU}}(x) = \max(0, x) = \begin{cases} 0, & x < 0, \\ x, & x \geq 0. \end{cases} \quad (6)$$

The ReLU activation function is illustrated in Fig. 2.

ReLU became popular because of its simplicity and computational efficiency. For negative inputs, the output is zero, whereas for positive inputs the function behaves linearly. This piecewise behavior helps mitigate the vanishing-gradient problem that affected earlier activation functions such as the sigmoid and hyperbolic tangent functions when training very deep networks.

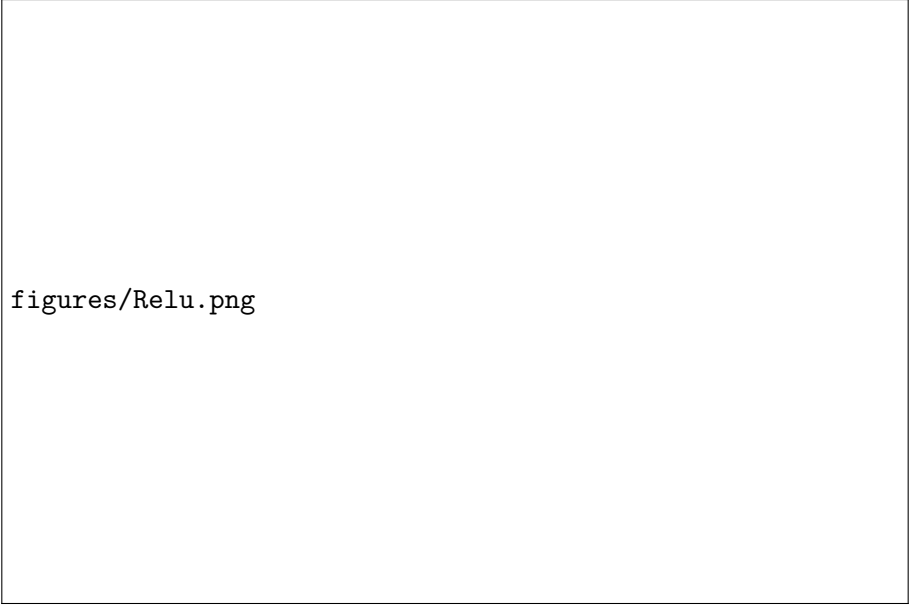
Although ReLU has become the standard activation function in many machine learning applications such as image classification, natural language processing, and computer vision, it is generally not the preferred choice for Physics-Informed Neural Networks (PINNs).

The reason is that PINNs require automatic differentiation to compute first- and higher-order derivatives of the neural network output with respect to the input variables. Since the ReLU function is not differentiable at $x = 0$ and has a zero second derivative almost everywhere, it often produces poor approximations of differential operators appearing in partial differential equations.

For this reason, smooth activation functions such as the hyperbolic tangent function,

$$\sigma(x) = \tanh(x), \quad (7)$$

are commonly preferred in PINNs. Their smoothness allows accurate computation of higher-order derivatives and generally leads to better convergence when solving differential equations.



figures/Relu.png

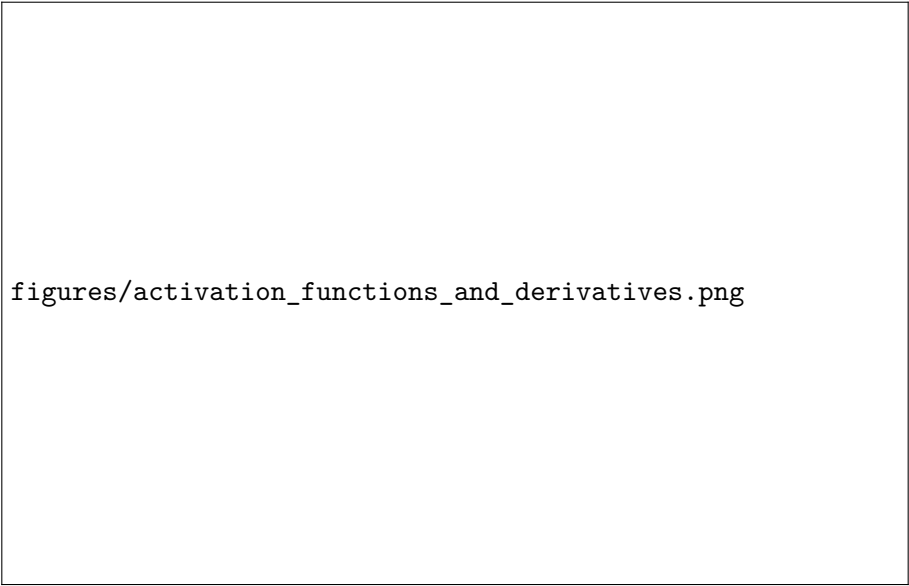
Figure 2: The Relu function

In this work, the hyperbolic tangent activation function is primarily employed:

$$\sigma(z) = \tanh(z) \tag{8}$$

The `tanh` activation is commonly used in PINN applications because:

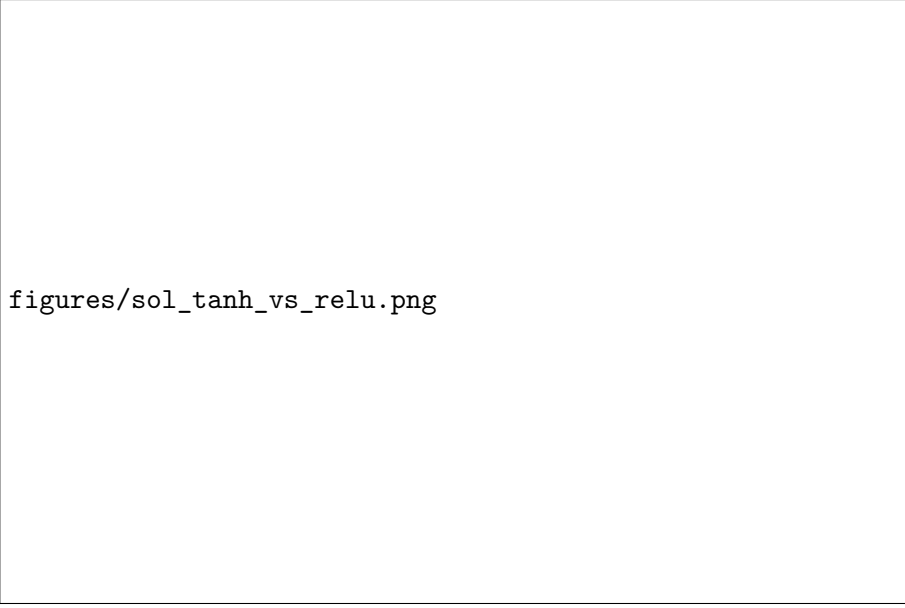
- it is smooth and infinitely differentiable,
- its derivatives are stable for automatic differentiation,
- and it generally provides better convergence properties for PDE-related problems compared to piecewise linear activations such as ReLU.



figures/activation_functions_and_derivatives.png

Figure 3: Comparison of the derivatives of the `tanh` and `swish` activation functions.

A combined visualization of the activation functions and their derivatives is shown in Figure ??.



figures/sol_tanh_vs_relu.png

Figure 4: Comparison of the derivatives of the `tanh` and `swish` activation functions.

Since PINNs require repeated computation of spatial and temporal derivatives through automatic differentiation, smooth activation functions are particularly important.

2.4 Automatic Differentiation

A central component of the PINN framework is automatic differentiation. Rather than approximating derivatives numerically using finite differences, derivatives are computed analytically through the computational graph of the neural network.

This allows direct evaluation of PDE residuals such as:

$$R(x, t) = \frac{\partial T_\theta}{\partial t} - \alpha \frac{\partial^2 T_\theta}{\partial x^2} \quad (9)$$

Automatic differentiation provides highly accurate derivative evaluations and enables efficient optimization of physics-based constraints.

In this work, automatic differentiation is performed using the JAX framework.

2.5 Weight Initialization Strategy

The initialization of neural network parameters plays a crucial role in the training dynamics and convergence behavior of Physics-Informed Neural Networks (PINNs).

Poor initialization may lead to:

- vanishing gradients,
- exploding gradients,
- unstable optimization,
- slow convergence,
- or failure to satisfy the PDE constraints.

Because PINNs require repeated computation of higher-order derivatives through automatic differentiation, initialization becomes even more important than in standard supervised learning problems.

In this work, the network weights are primarily initialized using the Glorot uniform initialization method, also known as Xavier uniform initialization.

2.5.1 Glorot Uniform Initialization

The Glorot initialization strategy was introduced to maintain approximately constant variance of activations and gradients throughout the neural network layers during forward and backward propagation.

For a layer containing:

- n_{in} input neurons,
- and n_{out} output neurons,

the weights are initialized from a uniform distribution:

$$W \sim \mathcal{U} \left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}} \right) \quad (10)$$

where:

- $\mathcal{U}$ denotes the uniform distribution,
- and the bounds are chosen to preserve variance stability across layers.

This initialization helps prevent:

- signal amplification,
- signal attenuation,
- and unstable gradient propagation.

2.5.2 Importance for PINNs

The Glorot initialization is particularly well suited for PINNs using smooth activation functions such as `tanh`.

Since PINNs rely heavily on:

- automatic differentiation,
- PDE residual computation,
- and higher-order derivatives,

poor initialization may rapidly produce:

- nearly constant outputs,
- saturated activations,
- or unstable PDE residuals.

For example, if the weights are initialized with excessively large values, the `tanh` activation function may rapidly saturate:

$$\tanh(z) \rightarrow \pm 1 \quad (11)$$

leading to near-zero gradients and difficult optimization.

Conversely, excessively small initial weights may produce weak gradients and very slow learning.

The Glorot initialization provides a good compromise by maintaining stable activation amplitudes throughout the network during the early stages of training.

This is especially important for PINNs because:

- PDE residual optimization is already highly stiff,
- multiple loss terms must be balanced simultaneously,
- and inverse parameter estimation problems are particularly sensitive to optimization stability.

2.5.3 Alternative Initialization Methods

Several alternative initialization strategies exist and may also be used depending on the activation functions and problem characteristics.

He Initialization He initialization is commonly used for ReLU-based neural networks.

The weights are initialized according to:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right) \quad (12)$$

or its uniform equivalent.

This initialization is designed to compensate for the asymmetric behavior of ReLU activations.

However, ReLU activations are generally less suitable for PINNs because:

- they are not smooth,
- their second derivatives vanish almost everywhere,
- and PDE residual computation becomes less accurate.

LeCun Initialization LeCun initialization is often associated with activation functions such as SELU.

It uses:

$$W \sim \mathcal{N}\left(0, \frac{1}{n_{in}}\right) \quad (13)$$

This initialization can improve stability for certain self-normalizing architectures.

Random Small Initialization A simpler strategy consists of initializing all weights with small random values.

Although easy to implement, this approach often leads to:

- unstable convergence,
- poor gradient propagation,
- and inconsistent optimization behavior.

Such naive initialization methods are generally not robust enough for PINN applications.

2.5.4 Initialization and Optimization Dynamics

The experiments performed in this work suggest that initialization strongly influences:

- convergence speed,
- optimization stability,
- PDE residual minimization,
- and inverse parameter identifiability.

In some cases, different random initializations may lead the optimization process toward:

- different local minima,
- different reconstructed parameters,
- or different balances between physics and data losses.

This highlights the highly non-convex nature of PINN optimization landscapes.

Careful initialization therefore constitutes an important component of stable and reproducible PINN training strategies.

2.6 Collocation Points

2.7 Collocation Points and Sampling Strategies

One of the major advantages of Physics-Informed Neural Networks is their ability to enforce physical laws throughout the entire spatio-temporal domain, including regions where no experimental measurements are available. While observational data are generally limited to a finite number of measurement locations, the governing partial differential equation can be evaluated at any point of the domain through automatic differentiation.

To achieve this objective, PINNs introduce a set of collocation points distributed throughout the computational domain. These points are not associated with experimental observations. Instead, they are used to evaluate the residual of the governing equation and enforce physical consistency everywhere in the domain.

The residual loss is computed at each collocation point and penalizes violations of the governing equation. For a set of (N_{col}) collocation points $((x_i, t_i))$, the physics loss is defined as

$$\mathcal{L}_{phys} = \frac{1}{N_{col}} \sum_{i=1}^{N_{col}} R(x_i, t_i)^2, \quad (14)$$

where $(R(x_i, t_i))$ denotes the PDE residual evaluated at the collocation point $((x_i, t_i))$

The number and distribution of collocation points play a critical role in the performance of a PINN. In general, a sufficient number of points is required to ensure that the physical constraints are enforced throughout the domain. However, increasing the number of collocation points also increases the computational cost of training, since the residual and its derivatives must be evaluated at every collocation point during each optimization step.

Several strategies can be used to generate collocation points.

A first possibility consists of using a fixed structured grid. While simple to implement, this approach may lead to an excessive concentration of points along predefined locations. In some situations, the network may learn to satisfy the governing equation primarily at the grid locations while exhibiting poorer accuracy between them.

Purely random sampling generally improves the coverage of the domain and reduces this risk. However, random sampling can occasionally generate clusters of points in certain regions

while leaving other regions sparsely sampled, especially when the number of collocation points is limited.

To overcome this limitation, Latin Hypercube Sampling (LHS) is frequently employed in PINN applications. The method divides the domain into multiple subregions and ensures that samples are distributed more uniformly throughout the domain while preserving a degree of randomness. This typically leads to better coverage of the spatio-temporal domain and improved training stability.

In more challenging problems involving boundary layers, sharp gradients, interfaces, or complex geometries, adaptive sampling strategies may also be employed. In these approaches, additional collocation points are progressively introduced in regions where the PDE residual remains large, allowing the network to focus computational effort where it is most needed.

The choice of sampling strategy is also closely related to the optimization algorithm used during training. When using first-order optimizers such as Adam, collocation points can be regenerated periodically or even at every training epoch. Since Adam relies on stochastic gradient updates, it is relatively robust to changes in the sampled points and can benefit from exploring different regions of the domain throughout training.

In contrast, quasi-Newton optimizers such as L-BFGS assume a fixed optimization landscape during the line-search and Hessian approximation procedures. For this reason, the collocation points are typically kept fixed during the entire L-BFGS optimization phase. In this context, Latin Hypercube Sampling often provides an effective compromise between domain coverage and training stability.

The distribution of collocation points ultimately has a significant impact on the convergence behavior, physical consistency, and predictive accuracy of the resulting PINN solution.

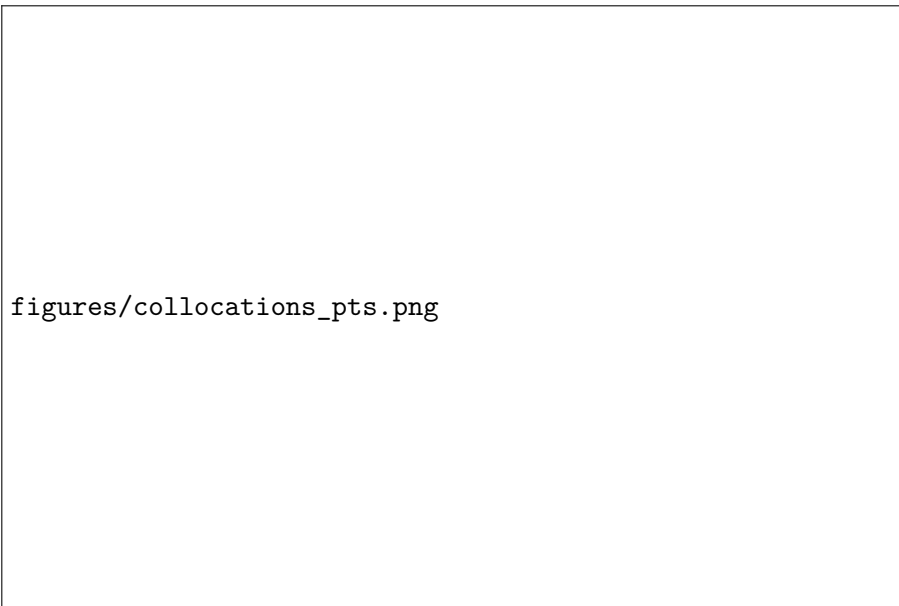


Figure 5: Comparison of common collocation point generation strategies. Uniform grids may lead to poor generalization between sampling locations, purely random sampling may produce clusters and empty regions, whereas Latin Hypercube Sampling provides a more uniform coverage of the spatio-temporal domain.

2.8 Boundary and Initial Condition Enforcement

A Physics-Informed Neural Network must not only satisfy the governing partial differential equation inside the domain, but also the initial and boundary conditions of the physical problem. For the one-dimensional heat diffusion problem considered in this work, the temperature field is

defined on the spatial domain $x \in [0, L]$ and the temporal domain $t \in [0, t_{\max}]$. The governing equation is

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad (15)$$

The associated initial and boundary conditions are

$$T(x, 0) = T_0(x), \quad (16)$$

and

$$T(0, t) = 0, \quad T(L, t) = 0. \quad (17)$$

In this work, the initial condition is chosen as

$$T_0(x) = \sin\left(\frac{\pi x}{L}\right). \quad (18)$$

The PINN must therefore produce a temperature field that is consistent with the data, satisfies the heat equation inside the domain, and respects the prescribed initial and boundary conditions.

Two main approaches can be used to impose initial and boundary conditions in PINNs:

- soft constraints,
- hard constraints.

2.8.1 Soft Constraint Formulation

In the soft constraint approach, the initial and boundary conditions are not imposed directly in the structure of the neural network. Instead, they are added as penalty terms in the loss function.

The neural network directly predicts the temperature field as

$$T_\theta(x, t) = \mathcal{N}_\theta(x, t). \quad (19)$$

The physics loss is computed from the PDE residual,

$$R(x, t) = \frac{\partial T_\theta}{\partial t} - \alpha \frac{\partial^2 T_\theta}{\partial x^2} \quad (20)$$

For a set of N_f collocation points (x_i, t_i) , the physics loss is written as

$$\mathcal{L}_{phys} = \frac{1}{N_f} \sum_{i=1}^{N_f} R(x_i, t_i)^2. \quad (21)$$

The initial condition loss is defined by comparing the network prediction at $t = 0$ with the prescribed initial profile:

$$\mathcal{L}_{IC} = \frac{1}{N_{IC}} \sum_{i=1}^{N_{IC}} [T_\theta(x_i, 0) - T_0(x_i)]^2. \quad (22)$$

Similarly, the boundary condition loss enforces the temperature at both ends of the domain:

$$\mathcal{L}_{BC} = \frac{1}{N_{BC}} \sum_{i=1}^{N_{BC}} [T_\theta(0, t_i)^2 - 0^2 + T_\theta(L, t_i)^2 - 0^2]. \quad (23)$$

If observational data are available, a data loss can also be added:

$$\mathcal{L}_{data} = \frac{1}{N_d} \sum_{i=1}^{N_d} \left[T_{\theta}(x_i, t_i) - T_i^{obs} \right]^2 \quad (24)$$

The total loss function is then written as

$$\mathcal{L}_{total} = \lambda_{data} \mathcal{L}_{data} + \lambda_{phys} \mathcal{L}_{phys} + \lambda_{IC} \mathcal{L}_{IC} + \lambda_{BC} \mathcal{L}_{BC}. \quad (25)$$

The coefficients λ_{data} , λ_{phys} , λ_{IC} , and λ_{BC} control the relative importance of each constraint during training.

This formulation is flexible and useful when the initial or boundary conditions are uncertain, noisy, or only partially known. However, it introduces a trade-off between the different loss terms. If the weighting coefficients are not properly chosen, the network may satisfy the PDE while poorly satisfying the boundary conditions, or conversely fit the initial and boundary conditions while producing a physically inconsistent solution inside the domain.

2.8.2 Hard Constraint Formulation

In the hard constraint approach, the initial and boundary conditions are embedded directly into the analytical form of the neural network output. Instead of letting the network freely predict $T_{\theta}(x, t)$, the prediction is constructed using an ansatz that automatically satisfies the prescribed conditions.

For the one-dimensional heat equation with

$$T(x, 0) = \sin\left(\frac{\pi x}{L}\right), \quad T(0, t) = 0, \quad T(L, t) = 0, \quad (26)$$

a suitable hard-constrained formulation is

$$T_{\theta}(x, t) = \sin\left(\frac{\pi x}{L}\right) + tx(L - x) \mathcal{N}_{\theta}(x, t). \quad (27)$$

This expression is commonly referred to as an ansatz. The ansatz is constructed so that the known constraints are automatically satisfied regardless of the values predicted by the neural network.

Indeed, at $t = 0$,

$$T_{\theta}(x, 0) = \sin\left(\frac{\pi x}{L}\right) + 0 \cdot x(L - x) \mathcal{N}_{\theta}(x, 0), \quad (28)$$

which gives

$$T_{\theta}(x, 0) = \sin\left(\frac{\pi x}{L}\right). \quad (29)$$

The initial condition is therefore satisfied exactly.

Similarly, at $x = 0$,

$$T_{\theta}(0, t) = \sin(0) + t \cdot 0 \cdot L \cdot \mathcal{N}_{\theta}(0, t) = 0, \quad (30)$$

and at $x = L$,

$$T_{\theta}(L, t) = \sin(\pi) + t \cdot L \cdot (L - L) \cdot \mathcal{N}_{\theta}(L, t) = 0 \quad (31)$$

The boundary conditions are therefore also satisfied exactly for every value of t .

Because the initial and boundary conditions are already enforced by construction, the corresponding loss terms are no longer required. The total loss function becomes

$$\mathcal{L}_{total} = \lambda_{data}\mathcal{L}_{data} + \lambda_{phys}\mathcal{L}_{phys}. \quad (32)$$

The optimization process can therefore focus entirely on minimizing the data loss and the physics residual.

2.8.3 Comparison Between Soft and Hard Constraints

Soft constraints are simple to implement and provide flexibility when the prescribed conditions are uncertain or noisy. However, they require careful tuning of multiple weighting coefficients and may not enforce the conditions with high accuracy.

Hard constraints require the construction of an appropriate ansatz adapted to the problem under consideration. Although this introduces additional analytical work, the initial and boundary conditions are satisfied exactly throughout training.

For benchmark problems where the conditions are known precisely, hard constraints generally improve numerical accuracy, reduce the number of competing loss terms, and simplify the optimization problem. For this reason, the hard-constrained formulation is adopted throughout most of the present study.

2.9 Optimization Strategy

Training a PINN corresponds to solving a high-dimensional nonlinear optimization problem.

In this work, optimization is primarily performed using the Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) algorithm implemented through the `jaxopt` library.

L-BFGS is particularly attractive for PINN applications because:

- it provides fast convergence on smooth optimization problems,
- it efficiently exploits second-order curvature information,
- and it often converges more reliably than stochastic gradient-based optimizers for PDE-constrained learning.

The optimization process iteratively updates the network parameters in order to minimize the global loss function.

2.10 Floating Point Precision

PINNs can be sensitive to numerical precision, especially when computing higher-order derivatives or balancing multiple loss terms with different magnitudes.

For this reason, computations in this work are performed using double precision (`float64`) rather than single precision (`float32`).

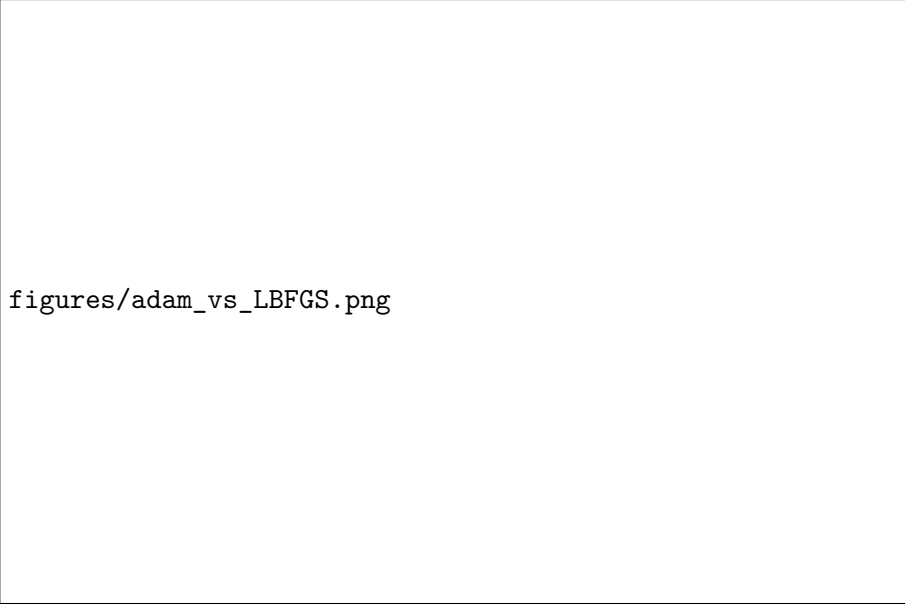
This improves:

- numerical stability,
- derivative accuracy,
- and optimization robustness.

2.11 Random Seeds and Reproducibility

Reproducibility is an important aspect of Physics-Informed Neural Network (PINN) research. Unlike deterministic numerical solvers, PINN training involves several stochastic components that may significantly influence:

- convergence behavior,



figures/adam_vs_LBFGS.png

Figure 6: Comparison between `adam` and LBFGS optimizers

- optimization stability,
- final solution accuracy,
- and inverse parameter reconstruction.

For this reason, careful management of random seeds is essential in order to obtain reproducible and interpretable results.

In this work, several independent sources of randomness were identified and controlled separately.

2.11.1 Weight Initialization Seed

The first important source of randomness concerns neural network weight initialization.

Since the network parameters are initialized randomly using methods such as Glorot uniform initialization, different seeds produce different initial optimization trajectories.

This may strongly affect:

- convergence speed,
- local minima selection,
- PDE residual minimization,
- and inverse parameter estimation.

Even when all hyperparameters remain identical, two different initialization seeds may lead to significantly different final solutions due to the highly non-convex nature of the PINN optimization landscape.

For this reason, the initialization seed must always be controlled and reported.

2.11.2 Collocation Point Seed

A second important source of randomness concerns the generation of collocation points used to evaluate the PDE residual.

In this work, collocation points are typically generated using Latin Hypercube Sampling (LHS).

Different seeds therefore produce different spatial and temporal point distributions inside the domain.

This directly influences:

- PDE residual evaluation,
- domain coverage,
- optimization stability,
- and parameter identifiability.

For inverse problems, collocation point placement can become particularly important because:

- some regions contain significantly more information regarding the unknown parameter α ,
- while other regions may contribute little useful information.

Consequently, collocation seeds may substantially affect inverse reconstruction quality.

2.11.3 Dataset Generation Seed

Another critical source of randomness concerns synthetic dataset generation.

Noisy observations are generated by adding random perturbations to the analytical solution:

$$T_{obs} = T_{exact} + \varepsilon \quad (33)$$

where ε represents random noise.

Different dataset seeds therefore generate different noisy realizations of the same physical problem.

This may influence:

- the signal-to-noise ratio,
- the effective information content,
- and the difficulty of the inverse problem.

In some configurations, specific noise realizations may strongly degrade parameter identifiability.

For this reason, robustness studies should not rely on a single dataset realization.

2.11.4 Training and Optimization Randomness

Additional stochasticity may also appear during optimization depending on:

- minibatch selection,
- hardware execution order,
- floating point precision,
- parallel execution,
- and framework-specific nondeterministic operations.

Although deterministic optimizers such as L-BFGS reduce part of this variability, numerical differences may still emerge depending on:

- compilation,
- execution backend,
- and machine architecture.

This is particularly relevant when using:

- GPU acceleration,
- JIT compilation,
- or highly optimized automatic differentiation libraries.

2.11.5 Importance of Multi-Seed Robustness Studies

The experiments performed in this work showed that PINN performance may vary significantly depending on the selected seeds.

A single successful run is therefore insufficient to evaluate the robustness of a PINN methodology.

For this reason, robustness studies were systematically conducted using multiple:

- initialization seeds,
- dataset seeds,
- and collocation point seeds.

This allows:

- estimation of variability across runs,
- identification of unstable training configurations,
- and more reliable evaluation of inverse reconstruction performance.

Such analyses are particularly important for inverse problems where optimization landscapes are highly non-convex and sensitive to small perturbations.

2.11.6 Seed Management in Practice

In practice, careful seed management requires:

- explicit initialization of random number generators,
- independent control of different randomness sources,
- and systematic recording of seed values for reproducibility.

Typical controlled seeds in this work include:

- neural network initialization seeds,
- collocation sampling seeds,
- noisy dataset generation seeds,
- and experiment-level global seeds.

This reproducibility strategy is essential for:

- fair comparison between experiments,
- reliable hyperparameter studies,
- and rigorous evaluation of PINN robustness.

2.12 Role of the Forward PINN Benchmark

Although the forward heat diffusion problem is relatively simple and classical numerical solvers remain significantly more efficient for this type of simulation, studying the forward PINN problem remains highly valuable.

The forward benchmark provides a controlled environment for understanding:

- how PINNs learn physical constraints,
- how optimization behaves under different hyperparameter choices,
- how data and physics losses interact,
- and how numerical pathologies may emerge during training.

This understanding is essential before addressing more challenging inverse problems or extending the methodology toward nonlinear PDEs, complex geometries, multiphysics systems, or real experimental datasets.

3 PINN Implementations and Computational Frameworks

The implementation of Physics-Informed Neural Networks (PINNs) strongly depends on the computational framework used to construct the neural network, compute automatic derivatives, and perform the optimization process. Several machine learning libraries are now commonly used for PINN development, each introducing different implementation philosophies, advantages, and technical difficulties.

In practice, implementing PINNs is often significantly more challenging than implementing standard supervised neural networks. PDE residual computation, higher-order automatic differentiation, inverse parameter optimization, and physics-data coupling introduce additional numerical and programming complexity.

This section summarizes the main frameworks investigated in this work and discusses some of the practical implementation challenges encountered during PINN development.

3.1 TensorFlow-Based Implementations

TensorFlow was one of the first major frameworks used for PINN research, and many foundational PINN papers were originally implemented using TensorFlow.

TensorFlow provides:

- automatic differentiation,
- GPU acceleration,
- scalable optimization tools,
- and compatibility with scientific machine learning workflows.

However, implementing PINNs in TensorFlow can rapidly become technically cumbersome, especially for inverse problems or custom optimization strategies.

One important difficulty arises when using external optimizers such as L-BFGS. TensorFlow neural network parameters are naturally stored as lists of tensors with different shapes, while many scientific optimizers expect a single flattened parameter vector.

As a result, TensorFlow PINN implementations often require:

- flattening all neural network weights into a single vector,
- unpacking this vector back into tensor structures,
- manually managing parameter reshaping,
- and carefully synchronizing trainable variables during optimization.

This pack/unpack procedure considerably complicates the implementation compared to standard neural network training.

Additional difficulties include:

- verbose gradient tape management,
- cumbersome higher-order derivative computation,
- graph debugging complexity,
- and numerical instability for stiff PDE-constrained optimization problems.

Although TensorFlow remains powerful, PINN implementations can become relatively heavy and difficult to maintain for research-oriented experimentation.

3.2 PyTorch-Based Implementations

PyTorch became increasingly popular in the scientific machine learning community due to its dynamic computational graph and more intuitive programming style.

Compared to TensorFlow, PyTorch generally offers:

- simpler debugging,
- cleaner syntax,
- easier prototyping,
- and greater flexibility for custom architectures and training loops.

Automatic differentiation is handled through the `autograd` engine, which considerably simplifies derivative computations required for PDE residuals.

However, PINN implementations in PyTorch still introduce several technical challenges.

Higher-order derivatives required for PDE residuals often require:

- repeated graph retention,
- nested automatic differentiation calls,
- and careful memory management.

For example, computing second-order spatial derivatives may require setting:

$$\text{create_graph}=\text{True} \tag{34}$$

during differentiation in order to preserve the computational graph for subsequent derivative evaluations.

As a consequence:

- memory usage can increase significantly,
- training can become computationally expensive,
- and debugging gradient-related issues may remain difficult.

Inverse problems also introduce optimization difficulties, especially when simultaneously training neural network weights and physical parameters such as α .

Despite these challenges, PyTorch remains highly attractive for exploratory PINN research and rapid experimentation.

3.3 JAX-Based Implementations

In this work, particular attention was devoted to JAX-based implementations because of their strong compatibility with scientific computing and differentiable programming.

JAX combines:

- NumPy-like syntax,
- just-in-time (JIT) compilation,
- efficient vectorization,
- and highly optimized automatic differentiation.

JAX is particularly attractive for PINNs because higher-order derivatives can often be implemented more naturally and efficiently than in other frameworks.

For example:

- PDE residuals can be vectorized efficiently,
- derivatives can be composed recursively,
- and large collocation point batches can be evaluated very efficiently.

However, JAX also introduces a very different programming philosophy that can initially be challenging.

Unlike TensorFlow or PyTorch, JAX requires a much more explicit and low-level implementation style. Many operations that are implicit in other frameworks must be manually defined.

For example, users often need to explicitly implement:

- parameter trees,
- forward passes,
- vectorization strategies,
- differentiation pipelines,
- and optimizer update rules.

Another important constraint is that JAX strongly encourages functional programming principles:

- arrays are immutable,
- functions should remain pure,
- and state management must be handled carefully.

This can initially make the implementation more difficult for users coming from traditional deep learning frameworks.

Debugging can also become complicated due to:

- JIT compilation,
- tracing behavior,
- and delayed execution mechanisms.

Nevertheless, once mastered, JAX provides extremely powerful tools for scientific machine learning, especially for PDE-constrained optimization and large-scale differentiable simulations.

3.4 Comparison Between Frameworks

Each framework presents different trade-offs between usability, flexibility, computational efficiency, and implementation complexity.

TensorFlow historically dominated early PINN research but often leads to relatively heavy implementations for inverse problems and custom optimizers.

PyTorch offers more intuitive coding and easier experimentation, although higher-order differentiation and memory management can still become difficult for complex PDEs.

JAX provides excellent differentiation and vectorization capabilities for scientific computing applications, but requires a more explicit and technically demanding implementation style.

In practice, the choice of framework depends on:

- the complexity of the PDE,
- the desired optimization strategy,
- computational performance requirements,
- implementation flexibility,
- and the developer’s familiarity with differentiable programming paradigms.

3.5 General Challenges in PINN Implementations

Regardless of the framework used, several numerical and implementation difficulties remain common to most PINN approaches:

- balancing physics and data losses,
- optimization stiffness,
- sensitivity to hyperparameters,
- efficient computation of higher-order derivatives,
- collocation point sampling strategies,
- and inverse parameter identifiability issues.

PINNs therefore require not only machine learning knowledge, but also a strong understanding of:

- numerical methods,
- PDEs,

- optimization,
- and scientific computing.

These implementation challenges partly explain why relatively simple benchmark problems remain highly valuable for developing intuition and understanding the practical behavior of PINNs before extending the methodology toward more realistic multiphysics systems.

4 One-Dimensional Heat Diffusion Problem

The physical problem considered in this work is the one-dimensional heat diffusion equation. This equation describes how temperature evolves in time within a medium due to thermal conduction. Despite its apparent simplicity, it captures many of the fundamental mechanisms encountered in diffusion-dominated physical systems and constitutes one of the most widely used benchmark problems for Physics-Informed Neural Networks (PINNs).

Studying this problem in detail provides a controlled mathematical framework where the behavior of PINNs can be analyzed rigorously before extending the methodology to more complex nonlinear or multiphysics systems.

4.1 Governing Equation

The one-dimensional heat equation is written as:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad (35)$$

where:

- $T(x, t)$ is the temperature field,
- x is the spatial coordinate,
- t is time,
- α is the thermal diffusivity coefficient.

The thermal diffusivity α characterizes the rate at which heat propagates through the material. Physically, large values of α correspond to faster temperature diffusion.

The spatial domain considered here is:

$$x \in [0, L] \quad (36)$$

with L the length of the domain.

4.2 Initial and Boundary Conditions

To obtain a well-posed problem, the governing equation must be supplemented with initial and boundary conditions.

The boundary conditions are homogeneous Dirichlet conditions:

$$T(0, t) = 0 \quad (37)$$

$$T(L, t) = 0 \quad (38)$$

which impose zero temperature at both ends of the domain.

The initial condition is chosen as:

$$T(x, 0) = \sin\left(\frac{\pi x}{L}\right) \quad (39)$$

This initial condition is particularly convenient because it is compatible with the boundary conditions and corresponds to a single spatial eigenmode of the diffusion operator.

4.3 Derivation of the Exact Solution

The exact analytical solution can be derived using the method of separation of variables.

We assume that the solution can be written as:

$$T(x, t) = X(x)\Theta(t) \quad (40)$$

where:

- $X(x)$ contains the spatial dependence,
- $\Theta(t)$ contains the temporal dependence.

Substituting this expression into the heat equation gives:

$$X(x)\frac{d\Theta}{dt} = \alpha\Theta(t)\frac{d^2X}{dx^2} \quad (41)$$

Dividing both sides by $X(x)\Theta(t)$ yields:

$$\frac{1}{\Theta}\frac{d\Theta}{dt} = \alpha\frac{1}{X}\frac{d^2X}{dx^2} \quad (42)$$

The left-hand side depends only on time while the right-hand side depends only on space. Therefore, both sides must be equal to the same constant, denoted $-\lambda$:

$$\frac{1}{\Theta}\frac{d\Theta}{dt} = -\lambda \quad (43)$$

$$\alpha\frac{1}{X}\frac{d^2X}{dx^2} = -\lambda \quad (44)$$

This leads to two ordinary differential equations.

4.3.1 Temporal Equation

$$\frac{d\Theta}{dt} = -\lambda\Theta \quad (45)$$

whose solution is:

$$\Theta(t) = Ae^{-\lambda t} \quad (46)$$

4.3.2 Spatial Equation

$$\frac{d^2X}{dx^2} + \frac{\lambda}{\alpha}X = 0 \quad (47)$$

Defining:

$$k^2 = \frac{\lambda}{\alpha} \quad (48)$$

the equation becomes:

$$\frac{d^2 X}{dx^2} + k^2 X = 0 \quad (49)$$

whose general solution is:

$$X(x) = B \sin(kx) + C \cos(kx) \quad (50)$$

Applying the boundary condition at $x = 0$:

$$X(0) = 0 \quad (51)$$

gives:

$$C = 0 \quad (52)$$

Thus:

$$X(x) = B \sin(kx) \quad (53)$$

Applying the second boundary condition at $x = L$:

$$X(L) = 0 \quad (54)$$

leads to:

$$\sin(kL) = 0 \quad (55)$$

which implies:

$$k_n = \frac{n\pi}{L} \quad (56)$$

for integer values $n = 1, 2, 3, \dots$

The associated eigenvalues are:

$$\lambda_n = \alpha \left(\frac{n\pi}{L} \right)^2 \quad (57)$$

The general solution of the heat equation is therefore:

$$T(x, t) = \sum_{n=1}^{\infty} A_n \sin \left(\frac{n\pi x}{L} \right) \exp \left[-\alpha \left(\frac{n\pi}{L} \right)^2 t \right] \quad (58)$$

4.4 Particular Solution Used in This Work

In this study, the initial condition corresponds exactly to the first eigenmode:

$$T(x, 0) = \sin \left(\frac{\pi x}{L} \right) \quad (59)$$

Therefore, only the first term of the series is present, yielding the exact solution:

$$T(x, t) = \sin \left(\frac{\pi x}{L} \right) \exp \left[-\alpha \left(\frac{\pi}{L} \right)^2 t \right] \quad (60)$$

This analytical expression is particularly useful because:

- it satisfies the governing equation exactly,
- it satisfies the boundary and initial conditions,

- it allows direct generation of synthetic datasets,
- and it provides an exact reference solution for evaluating PINN predictions and inverse parameter estimation performance.

5 Forward Problem

The first stage of this work focuses on the forward formulation of the one-dimensional heat diffusion equation. In the forward problem, the thermal diffusivity coefficient α is assumed to be known, and the objective is to reconstruct the temperature field $T(x, t)$ satisfying the governing partial differential equation together with the prescribed initial and boundary conditions.

In the context of Physics-Informed Neural Networks (PINNs), the solution is approximated by a neural network whose training is guided by the physics of the problem. More specifically, the network is trained such that its predictions satisfy the heat equation while also respecting the imposed conditions on the boundaries and at the initial time.

The PINN framework allows different training configurations. In a purely physics-driven setting, the model can be trained only from the residual of the governing equation and the associated constraints, without requiring any observational data. Alternatively, synthetic or experimental data points may also be incorporated into the loss function in order to guide the training process and improve convergence.

The loss function typically combines several contributions:

- a physics loss enforcing the partial differential equation,
- an initial condition loss,
- a boundary condition loss,
- and optionally a data loss term when observations are available.

For the one-dimensional heat equation, the physics residual is defined as:

$$R(x, t) = \frac{\partial T}{\partial t} - \alpha \frac{\partial^2 T}{\partial x^2} \quad (61)$$

The PINN is therefore optimized such that the residual approaches zero over the spatio-temporal domain.

Although the forward problem is an important benchmark for validating the implementation and understanding the training dynamics of PINNs, it should be noted that PINNs generally do not outperform classical numerical solvers for this type of problem. Traditional methods such as finite difference, finite volume, or finite element methods are typically faster, more accurate, and computationally more efficient for standard forward simulations of diffusion equations.

Consequently, the main interest of studying the forward problem within the PINN framework is not necessarily to replace classical solvers, but rather to better understand how neural networks learn physical constraints and how the optimization process behaves under different training conditions.

In particular, the forward problem provides a controlled environment for investigating:

- the balance between physics and data losses,
- convergence properties of the optimization process,
- the influence of collocation point distributions,
- the effect of network architecture and hyperparameters,

- and the role of automatic differentiation in enforcing physical laws.

This analysis constitutes an essential preliminary step before addressing the inverse problem, which is generally more challenging and represents one of the main motivations for using PINNs in scientific machine learning applications.

5.1 Normalization Strategies

An important aspect investigated during this work concerns the normalization of the physical variables and governing equations.

Normalization is commonly employed in Physics-Informed Neural Networks in order to:

- improve numerical stability,
- reduce scale disparities between variables,
- accelerate optimization,
- and improve gradient propagation during training.

However, for inverse problems involving unknown physical parameters, normalization may also unintentionally modify the sensitivity of the optimization process and introduce hidden dependencies between variables and trainable parameters.

For this reason, several normalization strategies were explored and compared throughout this work.

5.1.1 Fully Non-Dimensionalized Formulation

The first approach consists of fully non-dimensionalizing both space and time variables using characteristic scales.

Defining:

$$x^* = \frac{x}{L} \quad (62)$$

and

$$t^* = \frac{\alpha t}{L^2} \quad (63)$$

the heat equation becomes:

$$\frac{\partial T}{\partial t^*} = \frac{\partial^2 T}{\partial x^{*2}} \quad (64)$$

In this formulation, the diffusion coefficient α disappears from the PDE.

Although this normalization is mathematically elegant and commonly used in classical physics, it becomes problematic for inverse PINN problems because the parameter to identify no longer explicitly appears inside the governing equation.

As a consequence:

- the optimization may lose direct sensitivity to α ,
- parameter identifiability may deteriorate,
- and the inverse problem becomes artificially modified.

This formulation was therefore found to be unsuitable for the present inverse identification problem.

5.1.2 Partially Normalized Formulation

A second strategy consists of normalizing only the spatial and temporal coordinates independently from α .

For example:

$$x^* = \frac{x}{L} \quad (65)$$

$$t^* = \frac{t}{t_{max}} \quad (66)$$

In this case, the governing equation becomes:

$$\frac{\partial T}{\partial t^*} = \alpha \frac{t_{max}}{L^2} \frac{\partial^2 T}{\partial x^{*2}} \quad (67)$$

Unlike the fully non-dimensionalized formulation, α remains explicitly present in the PDE residual.

This approach improves:

- numerical conditioning,
- optimization stability,
- and input scaling consistency,

while preserving the inverse identification structure of the problem.

However, the scaling coefficients introduced in the PDE residual still modify the relative magnitude of the different loss terms and may influence optimization dynamics.

5.1.3 Non-Normalized Physical Formulation

A third approach consists of directly solving the PDE using the original physical variables without normalization.

The governing equation remains:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad (68)$$

Although this formulation may initially appear less numerically convenient, it presents several important advantages for inverse parameter estimation.

Most importantly:

- the physical meaning of the PDE is fully preserved,
- α remains directly coupled to the diffusion dynamics,
- and no artificial scaling modifies the identifiability structure of the inverse problem.

During the experiments performed in this work, the non-normalized formulation often provided more interpretable and physically consistent optimization behavior for inverse parameter reconstruction.

In particular, it avoided introducing hidden dependencies between:

- normalization scales,
- PDE residual magnitudes,
- and parameter sensitivities.

5.1.4 Influence of Normalization on Inverse Problems

These experiments highlighted an important observation:

Normalization strategies that are beneficial for forward problems are not necessarily appropriate for inverse PINN problems.

For forward simulations, removing physical scales may improve optimization efficiency without fundamentally altering the problem.

However, for inverse problems:

- the unknown parameters themselves define part of the physical scales,
- and normalization may unintentionally suppress informative parameter dependencies.

This becomes particularly important when studying:

- parameter identifiability,
- sensitivity analysis,
- noisy inverse reconstruction,
- and adaptive weighting strategies.

The normalization strategy therefore constitutes an important design choice in inverse PINN implementations and should be selected carefully depending on the objectives of the study.

6 Inverse Problem

While the forward heat diffusion problem consists of predicting the temperature evolution from known physical parameters, the inverse problem is fundamentally more challenging. In the inverse formulation, the objective is no longer only to compute the solution of the governing equation itself, but rather to infer unknown physical quantities directly from partial and potentially noisy observations. In this work, the inverse problem focuses on estimating the thermal diffusivity coefficient α from sparse temperature measurements distributed in space and time. Inverse problems are particularly important in science and engineering because many real-world situations involve unknown physical parameters that cannot be measured directly. Instead, these quantities must be reconstructed indirectly from experimental observations.

Examples include:

- thermal property identification, e.g., estimating the thermal diffusivity or conductivity of a material from temperature measurements;
- material characterization, e.g., identifying elastic moduli, permeability, or constitutive-law parameters from experimental data;
- medical imaging, e.g., reconstructing tissue properties or internal structures from MRI, ultrasound, or computed tomography measurements;
- fluid dynamics parameter estimation, e.g., estimating viscosity, boundary conditions, or unknown source terms from velocity and pressure data;
- geophysical inversion, e.g., reconstructing subsurface properties such as seismic-wave velocity or permeability from surface measurements;
- multiphysics system calibration, e.g., estimating coupled thermal, mechanical, or electromagnetic parameters from observations of the system.

Traditional numerical approaches for inverse problems often rely on repeated calls to classical PDE solvers inside iterative optimization loops. While these methods can be highly accurate, they may become computationally expensive when the parameter space is large, the governing equations are nonlinear, the geometry is complex, or experimental data are sparse and noisy. Physics-Informed Neural Networks (PINNs) provide an alternative framework where the physical solution and the unknown parameters can be learned simultaneously within a single optimization process. This constitutes one of the main motivations behind PINNs and scientific machine learning approaches.

6.1 Why PINNs Are Particularly Interesting for Inverse Problems

Unlike standard forward simulations, inverse problems represent a domain where PINNs may offer significant advantages compared to traditional numerical solvers. The neural network approximates the solution field, while unknown physical parameters are optimized simultaneously during training. For the present 1D thermal diffusion problem, the thermal diffusivity coefficient α becomes a trainable parameter of the optimization process. The inverse PINN therefore attempts to minimize both discrepancies with observational data, and violations of the governing PDE.

This allows PINNs to naturally integrate:

- sparse observations, e.g., temperature measurements available at only a limited number of spatial and temporal locations;
- noisy experimental measurements, e.g., temperature data affected by sensor noise or measurement uncertainty;
- physical constraints, e.g., governing equations, initial conditions, boundary conditions, or conservation laws;
- parameter estimation, e.g., identifying an unknown thermal diffusivity α from sparse and noisy temperature measurements.

within a unified differentiable framework. In contrast with many traditional inverse approaches, PINNs do not necessarily require repeated external PDE solves during optimization, since the PDE residual is directly embedded inside the loss function through automatic differentiation. However, although PINNs are conceptually attractive for inverse problems, they also introduce important optimization and identifiability challenges.

6.2 Inverse Problem Formulation

The inverse problem considered in this work consists of estimating the thermal diffusivity α from noisy temperature observations. Since no physical experiments were conducted, synthetic temperature measurements were generated using the following analytical solution:

$$T_{\text{exact}}(x, t) = \sin\left(\frac{\pi x}{L}\right) \exp\left[-\alpha\left(\frac{\pi}{L}\right)^2 t\right]. \quad (69)$$

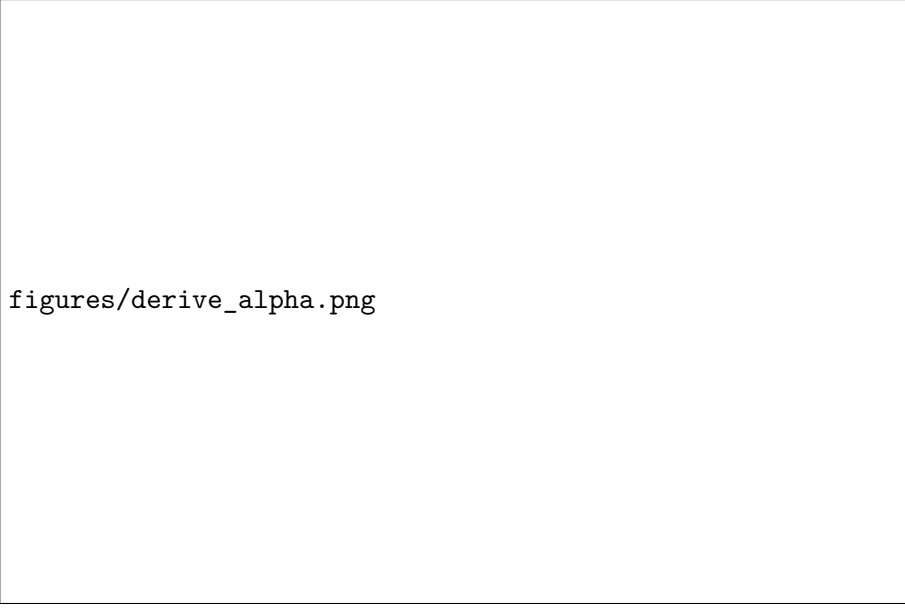
To simulate experimental measurement uncertainty, additive Gaussian noise was introduced:

$$T_{\text{obs},i} = T_{\text{exact}}(x_i, t_i) + \varepsilon_i, \quad (70)$$

where

$$\varepsilon_i \sim \mathcal{N}(0, \sigma^2), \quad (71)$$

and σ denotes the standard deviation of the measurement noise. As mentioned earlier, the PINN must therefore simultaneously:



figures/derive_alpha.png

Figure 7: Informative choice of alpha

- reconstruct the temperature field,
- satisfy the governing PDE,
- and infer the correct value of α .

6.3 Soft and Hard Constraint Strategies

6.4 Enforcement of Initial and Boundary Conditions

An important aspect investigated in this work is the enforcement of the initial and boundary conditions. Two strategies were explored: soft and hard constraint enforcement.

Soft constraints In the soft-constrained formulation, the initial and boundary conditions are enforced by adding penalty terms to the total loss function:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{physics}}\mathcal{L}_{\text{physics}} + \lambda_{\text{data}}\mathcal{L}_{\text{data}} + \lambda_{\text{initial}}\mathcal{L}_{\text{initial}} + \lambda_{\text{boundary}}\mathcal{L}_{\text{boundary}}. \quad (72)$$

This approach is straightforward and flexible because it does not require the construction of a particular analytical form for the network output. It can therefore be applied relatively easily to different initial and boundary conditions, including problems involving complex geometries. Soft constraints are also useful when the prescribed conditions are uncertain or affected by measurement noise, since potentially inaccurate values are not imposed exactly. Moreover, the relative importance of each condition can be adjusted through its weighting coefficient. However, this flexibility introduces several limitations:

- the initial and boundary conditions are generally satisfied only approximately;
- the different components of the loss function must be appropriately balanced and therefore the training results may be sensitive to the weighting coefficients λ_{initial} and $\lambda_{\text{boundary}}$;
- competing loss terms may lead to optimization difficulties and slower convergence.

Hard constraints In the hard-constrained formulation, the predicted solution is analytically constructed so that the initial and boundary conditions are satisfied exactly for any values of the neural-network parameters. For the one-dimensional heat equation considered in this work, the constrained solution is written as

$$T_\theta(x, t) = \sin\left(\frac{\pi x}{L}\right) + t x(L - x) N_\theta(x, t), \quad (73)$$

where $N_\theta(x, t)$ is the unconstrained output of the neural network. At $t = 0$, the second term vanishes, and the prescribed initial condition $T_\theta(x, 0) = \sin\left(\frac{\pi x}{L}\right)$ is recovered. Similarly, the factor $x(L - x)$ vanishes at $x = 0$ and $x = L$, ensuring that the homogeneous $T_\theta(0, t) = T_\theta(L, t) = 0$ boundary conditions are satisfied. This formulation offers several advantages:

- the initial and boundary conditions are satisfied exactly by design;
- the corresponding penalty terms and weighting coefficients are removed from the loss function and so the optimization problem contains fewer competing objectives;
- physical consistency, optimization stability, and convergence robustness can be improved.

Nevertheless, hard constraints also have limitations:

- an appropriate constrained analytical form must be derived for the specific problem;
- this construction may become difficult for complex geometries, mixed or time-dependent boundary conditions, and multiphysics problems;
- inaccurate or noisy prescribed conditions are imposed exactly, which may introduce bias into the predicted solution;
- the chosen transformation can affect the scaling and optimization of the neural-network correction.

In the present problem, the geometry and prescribed conditions are sufficiently simple to construct an explicit hard-constrained solution. This strategy was therefore progressively adopted to reduce the number of loss terms and improve the robustness of the inverse identification procedure. However, soft constraints remain more flexible and may be preferable when the conditions are uncertain or when an exact constrained formulation is difficult to derive.

6.5 Low-Amplitude Solutions and Parameter Identifiability

One of the main challenges encountered in the inverse PINN framework was the loss of identifiability of the thermal diffusivity α when the predicted temperature field becomes small. For the one-dimensional heat equation,

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}, \quad (74)$$

the physics residual is defined as

$$R_\theta(x, t; \alpha) = \frac{\partial T_\theta}{\partial t} - \alpha \frac{\partial^2 T_\theta}{\partial x^2}. \quad (75)$$

The identically zero field,

$$T_\theta(x, t) = 0, \quad (76)$$

is a solution of the homogeneous heat equation for any value of α . Consequently, if the initial and boundary conditions are not sufficiently enforced and the observational data do not

prevent it, the network may approach a low-amplitude solution that produces a small physics loss without correctly identifying α .

It should be emphasized that a small temperature amplitude does not, in general, imply small temporal or spatial derivatives. A low-amplitude function may still exhibit large derivatives, particularly if it contains rapid variations. However, for the analytical solution considered in this work,

$$T_{\text{exact}}(x, t) = \sin\left(\frac{\pi x}{L}\right) \exp\left[-\alpha\left(\frac{\pi}{L}\right)^2 t\right], \quad (77)$$

both relevant derivatives are directly proportional to the temperature field:

$$\frac{\partial T_{\text{exact}}}{\partial t} = -\alpha\left(\frac{\pi}{L}\right)^2 T_{\text{exact}}, \quad (78)$$

and

$$\frac{\partial^2 T_{\text{exact}}}{\partial x^2} = -\left(\frac{\pi}{L}\right)^2 T_{\text{exact}}. \quad (79)$$

Therefore, as the solution decays, its temporal derivative and spatial curvature also decrease. This progressively reduces the information available for estimating α .

Indeed, the sensitivity of the residual to the unknown parameter is

$$\frac{\partial R_\theta}{\partial \alpha} = -\frac{\partial^2 T_\theta}{\partial x^2}. \quad (80)$$

When the spatial curvature becomes small, changes in α have little influence on the residual. The physics loss may then remain small over a broad range of α values, leading to a poorly conditioned parameter identification problem.

This difficulty becomes particularly important for large values of the true diffusivity. In this regime, the temperature field decays rapidly and late-time observations contain little information about α . As a consequence:

- the signal amplitude and spatial curvature decrease rapidly;
- the loss becomes less sensitive to the value of α ;
- the signal-to-noise ratio deteriorates and different values of α may produce temperature predictions that are difficult to distinguish within the measurement noise.

This behavior highlights an important distinction between solution reconstruction and parameter identification. A PINN may reconstruct a low-amplitude temperature field with a small prediction error while still providing an inaccurate estimate of α . Accurate parameter recovery therefore requires observations in regions of the space-time domain where the solution remains sufficiently sensitive to the unknown diffusivity.

6.5.1 Parameter Reparameterization

The thermal diffusivity must satisfy

$$\alpha > 0, \quad (81)$$

because heat flows from hotter to colder regions. A negative diffusivity would instead describe an unphysical anti-diffusion process. Directly optimizing α does not guarantee that this physical constraint will be respected during training. Gradient-based updates may lead to negative or unrealistically small values, particularly during the early stages of optimization, when the

temperature field has not yet been accurately reconstructed. To enforce a positive lower bound, a new trainable variable β is introduced such that

$$\alpha = \alpha_{\text{floor}} + \beta^2, \quad (82)$$

where $\alpha_{\text{floor}} > 0$ is a prescribed minimum admissible value. The optimizer updates β , while α is reconstructed from this expression during each evaluation of the loss function. This guarantees that

$$\alpha \geq \alpha_{\text{floor}} > 0. \quad (83)$$

The effect of this transformation on the optimization dynamics can be understood using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \beta} = \frac{\partial \mathcal{L}}{\partial \alpha} \frac{d\alpha}{d\beta} = 2\beta \frac{\partial \mathcal{L}}{\partial \alpha}. \quad (84)$$

As α approaches α_{floor} , $|\beta|$ becomes small and the gradient transmitted to β is progressively reduced. The evolution of the physical parameter therefore slows down near its lower bound. This can prevent α from decreasing too rapidly during the early stages of training and give the neural network more time to reconstruct the temperature field. However, the square transformation also has limitations. If β becomes very close to zero, the gradient with respect to β may become extremely small, making it difficult for α to move away from its lower bound. Moreover, the mapping is not one-to-one, since β and $-\beta$ correspond to the same value of α . The reparameterization guarantees physical admissibility and modifies the parameter-learning dynamics, but it does not guarantee that the correct value of α will be identified. Other approaches may also be used to constrain the unknown parameter:

- **Softplus reparameterization:**

$$\alpha = \alpha_{\text{floor}} + \text{softplus}(\beta), \quad (85)$$

which provides a smooth positive mapping and avoids the zero derivative of the square transformation at $\beta = 0$;

- **Exponential reparameterization:**

$$\alpha = \alpha_{\text{floor}} + \exp(\beta), \quad (86)$$

which guarantees positivity but can generate very large parameter values and gradients;

- **Bounded sigmoid reparameterization:**

$$\alpha = \alpha_{\text{min}} + (\alpha_{\text{max}} - \alpha_{\text{min}}) \text{sigmoid}(\beta), \quad (87)$$

which is appropriate when physically justified lower and upper bounds are known, although gradient saturation may occur near these bounds;

- **Bound-constrained optimization:** the parameter is directly restricted to an admissible interval using projection or a constrained optimization algorithm;
- **Penalty-based regularization:** an additional term is introduced into the loss function to penalize negative or physically implausible values of α .

In the present work, the square reparameterization was selected because it is simple to implement, guarantees a positive diffusivity, and reduces rapid variations of α near the prescribed lower bound. Nevertheless, its effectiveness still depends on the initialization of β , the balance between the data and physics losses, and the sensitivity of the observations to α .

6.5.2 Sensitivity-Based Weighting

Even after introducing observational data and parameter reparameterization, inverse parameter identification remains challenging because the measurements do not all contain the same amount of information about the unknown diffusion coefficient α . Sensitivity-based weighting was therefore investigated to increase the influence of the observations that are the most informative for parameter identification.

For the analytical solution considered in this work,

$$T_{\text{exact}}(x, t) = \sin\left(\frac{\pi x}{L}\right) \exp\left[-\alpha \left(\frac{\pi}{L}\right)^2 t\right], \quad (88)$$

the sensitivity of the temperature to the diffusion coefficient is

$$S_{\alpha}(x, t) = \frac{\partial T_{\text{exact}}}{\partial \alpha} = -\left(\frac{\pi}{L}\right)^2 t \sin\left(\frac{\pi x}{L}\right) \exp\left[-\alpha \left(\frac{\pi}{L}\right)^2 t\right]. \quad (89)$$

The magnitude

$$|S_{\alpha}(x, t)| = \left| \frac{\partial T_{\text{exact}}}{\partial \alpha} \right| \quad (90)$$

measures the local variation of the temperature caused by a small change in α . A large value indicates that the observation is strongly affected by the diffusion coefficient and therefore contains useful information for its identification. Conversely, a small value indicates that substantially different values of α may produce similar temperature values at that location.

Weighted data loss The sensitivity weights are applied to the individual temperature-error terms in the data loss. For a set of N_d observations, the weighted data loss is defined as

$$\mathcal{L}_{\text{data}}^{(w)} = \frac{\sum_{i=1}^{N_d} w_i [T_{\theta}(x_i, t_i) - T_{\text{obs},i}]^2}{\sum_{i=1}^{N_d} w_i}. \quad (91)$$

Normalization by the sum of the weights keeps the scale of the data loss approximately independent of the selected weighting law and of the overall magnitude of the sensitivities. The weights therefore redistribute the relative contributions of the observations without arbitrarily changing the global scale of the data-fitting term.

Several sensitivity-based weighting laws were examined:

$$w_i = |S_{\alpha}(x_i, t_i)|, \quad (92)$$

$$w_i = S_{\alpha}(x_i, t_i)^2, \quad (93)$$

$$w_i = \sqrt{|S_{\alpha}(x_i, t_i)|}. \quad (94)$$

Quadratic weighting strongly concentrates the data loss on the observations with the largest sensitivities. This may reduce the effective use of the remaining dataset and make the optimization more sensitive to a limited number of points. Linear weighting produces a less concentrated distribution, but may still create a strong contrast between informative and weakly informative measurements.

In the experiments conducted here, the square-root formulation provided a useful compromise. It increased the contribution of informative observations while maintaining a broader participation of the available dataset.

Why sensitivity weights are applied only to the data loss The sensitivity weights are applied only to the data loss because they quantify the information carried by each temperature observation about the unknown parameter α . Their purpose is to redistribute the contribution of the measured data according to their relevance for parameter identification.

The other loss terms have fundamentally different roles. The physics loss is defined as

$$\mathcal{L}_{\text{physics}} = \frac{1}{N_f} \sum_{j=1}^{N_f} \left[\frac{\partial T_\theta}{\partial t}(x_j^f, t_j^f) - \alpha \frac{\partial^2 T_\theta}{\partial x^2}(x_j^f, t_j^f) \right]^2, \quad (95)$$

where (x_j^f, t_j^f) are the collocation points. This term enforces the heat equation throughout the space–time domain. Its objective is not to quantify the information content of an observation, but to ensure that the predicted temperature field is physically admissible.

Applying the same sensitivity weights to the physics residual would weaken the enforcement of the governing equation in regions where $|\partial T / \partial \alpha|$ is small. The PINN could consequently tolerate larger physics errors near the boundaries, at early times, or at late times, even though the heat equation remains valid and must be satisfied in all of these regions.

The same reasoning applies to possible initial- and boundary-condition losses. These terms impose the constraints that define the forward problem rather than measurements whose purpose is to identify α . In the present problem,

$$S_\alpha(x, 0) = 0, \quad S_\alpha(0, t) = S_\alpha(L, t) = 0. \quad (96)$$

Consequently, applying sensitivity weights to the initial-condition loss would assign zero importance to the prescribed initial temperature profile. Similarly, applying them to the boundary-condition loss would suppress the constraints at $x = 0$ and $x = L$. This could allow violations of the initial or boundary conditions precisely where they must be enforced.

The distinction can therefore be summarized as follows: sensitivity weighting determines which observations should influence parameter identification most strongly, whereas the physics, initial-condition, and boundary-condition terms define the set of physically admissible solutions. The total loss may thus be written as

$$\mathcal{L}_{\text{total}} = \lambda_{\text{data}} \mathcal{L}_{\text{data}}^{(w)} + \lambda_{\text{physics}} \mathcal{L}_{\text{physics}} + \lambda_{\text{IC}} \mathcal{L}_{\text{IC}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}}, \quad (97)$$

with the sensitivity weights applied only inside $\mathcal{L}_{\text{data}}^{(w)}$. When the initial and boundary conditions are imposed through a hard-constraint transformation, as in part of the present study, the corresponding losses are absent and the total loss reduces to

$$\mathcal{L}_{\text{total}} = \lambda_{\text{data}} \mathcal{L}_{\text{data}}^{(w)} + \lambda_{\text{physics}} \mathcal{L}_{\text{physics}}. \quad (98)$$

This choice does not imply that the other loss terms can never be weighted. Adaptive residual weighting or nonuniform collocation sampling may, for example, be used to improve the enforcement of the governing equation in difficult regions. Such physics-oriented strategies pursue a different objective and should be based on the residual, its derivatives, or an appropriate error indicator rather than automatically reusing weights designed to measure the parameter information contained in observations.

Physical interpretation of the sensitivity distribution For the present problem, the spatial and temporal distributions of sensitivity can be understood directly from the underlying physics. At the initial time,

$$S_\alpha(x, 0) = 0, \quad (99)$$

because the prescribed initial temperature profile is independent of α . The diffusion coefficient controls the subsequent evolution of the field, but not its initial value. Consequently, measurements taken exactly at $t = 0$ contain no direct information about α .

At very early times, diffusion has only weakly modified the initial temperature profile, so the effect of α remains limited. At late times, the temperature field has largely decayed and may become comparable to the measurement noise. Both the signal amplitude and its sensitivity to α then become small. Intermediate times are therefore generally the most informative: diffusion has acted sufficiently for the influence of α to become observable, while the temperature signal has not yet disappeared.

For a fixed spatial location, the sensitivity magnitude is proportional to

$$t \exp \left[-\alpha \left(\frac{\pi}{L} \right)^2 t \right]. \quad (100)$$

Differentiating this expression with respect to time shows that it reaches its maximum at

$$t_{\text{sens}} = \frac{L^2}{\alpha \pi^2}. \quad (101)$$

The most informative time therefore depends on the characteristic diffusion timescale and on the unknown parameter itself. If the observation interval ends before t_{sens} , the maximum sensitivity within the available dataset will instead occur at or near the latest measured time.

Spatially, the sensitivity magnitude is proportional to

$$\left| \sin \left(\frac{\pi x}{L} \right) \right|. \quad (102)$$

It vanishes at $x = 0$ and $x = L$, where the temperature is fixed by the homogeneous boundary conditions, and reaches its maximum at

$$x = \frac{L}{2}. \quad (103)$$

For the initial condition considered here, measurements near the center of the bar are therefore more informative about α than measurements near its boundaries. This spatial conclusion is specific to the present temperature mode and boundary conditions; a different initial condition or physical problem may produce a different sensitivity distribution.

Weighting when the exact solution is unavailable The analytical solution provides a controlled setting in which the exact sensitivity can be evaluated. In a real inverse problem, however, neither the exact solution nor the true value of α is generally known. The exact sensitivity map is therefore unavailable.

A first possibility is to define an approximate weighting function from physical reasoning. For the current problem, the preceding analysis suggests emphasizing the center of the domain and an intermediate time interval. A possible heuristic weight is

$$w(x, t) = w_{\min} + (1 - w_{\min}) \sin^2 \left(\frac{\pi x}{L} \right) \exp \left[-\frac{(t - t_c)^2}{2\sigma_t^2} \right], \quad (104)$$

where t_c represents an estimated informative time, σ_t controls the width of the selected interval, and $w_{\min} > 0$ prevents the remaining observations from being completely discarded.

This formulation encodes physical intuition but does not necessarily reproduce the optimal sensitivity distribution. Its parameters should therefore be validated numerically, and the resulting parameter estimate should be compared with the unweighted result.

When physical intuition is insufficient, the sensitivity can instead be approximated numerically around a nominal estimate $\hat{\alpha}$. One option is to solve two forward problems with slightly perturbed parameter values:

$$\alpha^+ = \hat{\alpha} + \Delta\alpha, \quad \alpha^- = \hat{\alpha} - \Delta\alpha. \quad (105)$$

The local sensitivity is then approximated by the centered finite difference

$$S_\alpha(x, t; \hat{\alpha}) \approx \frac{T(x, t; \hat{\alpha} + \Delta\alpha) - T(x, t; \hat{\alpha} - \Delta\alpha)}{2\Delta\alpha}. \quad (106)$$

The perturbation $\Delta\alpha$ must be sufficiently small to provide a local approximation, but not so small that numerical or training errors dominate the difference between the two solutions.

Another possibility is to solve the sensitivity equation. Defining

$$S(x, t) = \frac{\partial T(x, t)}{\partial \alpha}, \quad (107)$$

and differentiating the heat equation with respect to α gives

$$\frac{\partial S}{\partial t} = \alpha \frac{\partial^2 S}{\partial x^2} + \frac{\partial^2 T}{\partial x^2}. \quad (108)$$

If the initial and boundary conditions are independent of α , the corresponding sensitivity conditions are

$$S(x, 0) = 0, \quad S(0, t) = S(L, t) = 0. \quad (109)$$

This auxiliary equation can be solved together with the forward problem using a nominal or currently estimated value of α . It provides a systematic alternative to heuristic weighting and does not require knowledge of the exact solution.

Two-stage sensitivity-informed retraining A practical strategy for incorporating a numerically estimated sensitivity is to use a two-stage training procedure. In the first stage, the inverse PINN is trained without sensitivity-based weighting:

$$\left(\theta^{(1)}, \alpha^{(1)} \right) = \underset{\theta, \alpha}{\operatorname{argmin}} [\lambda_{\text{data}} \mathcal{L}_{\text{data}} + \lambda_{\text{physics}} \mathcal{L}_{\text{physics}}]. \quad (110)$$

This preliminary training provides an initial estimate $\alpha^{(1)}$. The sensitivity is subsequently approximated around this estimate, for example by solving two direct problems with

$$\alpha^\pm = \alpha^{(1)} \pm \Delta\alpha. \quad (111)$$

This additional computation is required because the network used in the present inverse formulation takes only (x, t) as inputs. Its output $T_\theta(x, t)$ does not depend explicitly on α . Consequently, $\partial T_\theta / \partial \alpha$ cannot be obtained by simply differentiating the network output with respect to the trainable parameter. The dependence of the learned field on α occurs indirectly through the optimization process and the physics loss.

The numerical sensitivity obtained from the perturbed direct solutions is

$$S_\alpha^{(1)}(x, t) \approx \frac{T(x, t; \alpha^{(1)} + \Delta\alpha) - T(x, t; \alpha^{(1)} - \Delta\alpha)}{2\Delta\alpha}. \quad (112)$$

Rather than using only the point at which the sensitivity is maximal, the complete sensitivity distribution is retained. Using only the maximum would concentrate the loss on a very restricted region and make the second training stage sensitive to noise and to errors in the preliminary estimate.

The sensitivity values at the observation locations are first normalized:

$$\hat{S}_i = \frac{|S_\alpha^{(1)}(x_i, t_i)|}{\max_j |S_\alpha^{(1)}(x_j, t_j)| + \varepsilon}, \quad (113)$$

where $\varepsilon > 0$ prevents division by zero. The weights can then be defined as

$$w_i = w_{\min} + (1 - w_{\min}) \hat{S}_i^p, \quad 0 < p \leq 1. \quad (114)$$

The lower bound $w_{\min}$ ensures that weakly sensitive observations still contribute to the loss. The exponent p controls the contrast between the weights. In particular, $p = 1/2$ corresponds to a square-root transformation that reduces excessive concentration around the maximum sensitivity.

Once calculated, these weights are treated as fixed coefficients during the second training stage. If they are computed using differentiable quantities derived from the current PINN prediction, their gradients should be stopped before they are inserted into the data loss. Otherwise, the optimization could partially reduce the weighted loss by modifying the weights themselves instead of reducing the temperature errors.

In the second stage, the inverse PINN is trained using the resulting weighted data loss:

$$\left(\theta^{(2)}, \alpha^{(2)} \right) = \underset{\theta, \alpha}{\operatorname{argmin}} \left[\lambda_{\text{data}} \mathcal{L}_{\text{data}}^{(w)} + \lambda_{\text{physics}} \mathcal{L}_{\text{physics}} \right]. \quad (115)$$

The second stage may be initialized from $(\theta^{(1)}, \alpha^{(1)})$, which corresponds to fine-tuning the preliminary solution. This approach is computationally efficient, but the result may remain dependent on the optimization trajectory of the first stage. Alternatively, the network may be restarted from a new initialization while retaining the calculated weights. This provides a stronger test of whether the sensitivity distribution itself improves parameter identification.

The procedure may also be repeated by recomputing the sensitivity around $\alpha^{(2)}$. However, only a limited number of cycles is advisable. If the initial estimate $\alpha^{(1)}$ is inaccurate, its sensitivity map may also be biased and could reinforce the initial identification error. Moderate weighting, a nonzero minimum weight, and systematic comparison with the unweighted solution are therefore important.

Overall, these experiments demonstrate that parameter-identification quality depends not only on the number of observations but also on the distribution of information within the space-time domain. Sensitivity weights are applied to the data loss because they measure the usefulness of observations for identifying α , whereas the other loss terms enforce the physical problem independently of this local information content.

When the exact sensitivity is unavailable, it may be approximated from a preliminary parameter estimate and incorporated through a two-stage sensitivity-informed retraining procedure. Sensitivity weighting can improve the use of informative measurements, but it cannot create information that is absent from the data and does not by itself guarantee correct parameter identification.

6.6 Noise Sensitivity and Identifiability Challenges

One of the main challenges encountered in the inverse problem is the identification of the diffusion coefficient α from sparse and noisy temperature observations. Even when the PINN reconstructs a visually accurate temperature field and produces a small physics residual, the estimated value of α may remain inaccurate.

This behavior illustrates an important distinction between forward-solution accuracy and parameter identifiability. Minimizing the residual of the governing equation ensures that the predicted field approximately satisfies

$$\frac{\partial T}{\partial t} - \alpha \frac{\partial^2 T}{\partial x^2} = 0, \quad (116)$$

but it does not necessarily guarantee that the correct value of α has been identified. Different combinations of the neural-network parameters θ and the diffusion coefficient α may produce temperature fields that fit the available observations and approximately satisfy the heat equation. This ambiguity becomes particularly important when the measurements contain little information about α .

Loss of identifiability in the rapid-diffusion regime A particularly difficult situation arises for large values of α . For the initial and boundary conditions considered in this work, the exact solution is

$$T_{\text{exact}}(x, t; \alpha) = \sin\left(\frac{\pi x}{L}\right) \exp\left[-\alpha \left(\frac{\pi}{L}\right)^2 t\right]. \quad (117)$$

The characteristic diffusion timescale is

$$t_d = \frac{L^2}{\alpha \pi^2}. \quad (118)$$

As α increases, t_d decreases and the temperature field rapidly decays toward zero. Consequently,

$$T(x, t) \approx 0 \quad (119)$$

over a large fraction of the observation interval.

In this regime,

- the measurable temperature amplitude becomes weak;
- the observations may become dominated by noise;
- several values of α may produce nearly indistinguishable temperature fields;
- and the inverse optimization problem becomes poorly conditioned.

A near-zero temperature field can also approximately satisfy the homogeneous boundary conditions and produce a small PDE residual. The optimization may therefore approach a near-trivial solution that is physically admissible but only weakly informative about the true diffusion coefficient.

This difficulty is directly related to the parameter sensitivity

$$S_\alpha(x, t) = \frac{\partial T_{\text{exact}}}{\partial \alpha} = -\left(\frac{\pi}{L}\right)^2 t \sin\left(\frac{\pi x}{L}\right) \exp\left[-\alpha \left(\frac{\pi}{L}\right)^2 t\right]. \quad (120)$$

At $t = 0$, the sensitivity is zero because the initial temperature profile is independent of α . At late times, the exponential decay makes both the temperature and its sensitivity small. The observations are therefore most informative over an intermediate time interval, as discussed in Section ??.

Combined influence of data scarcity and noise Parameter identification generally becomes more difficult when the number of observations decreases or when the noise level increases. In the present study, the robustness limits of the inverse PINN were investigated using datasets containing as few as $N_d = 25$ observations and relative noise levels as high as $\eta = 0.20$.

These two effects interact. With a large dataset, random measurement errors may partially compensate for one another. With only a small number of observations, a particular realization of the noise may instead introduce a significant bias in the apparent temporal decay of the temperature. Since α is inferred primarily from this decay rate, even a few unfavorably perturbed observations may substantially modify the estimated parameter.

The difficulty therefore depends not only on the nominal noise amplitude but also on the particular realization and spatial-temporal distribution of the noise. For example, large errors affecting observations near the center of the bar and at intermediate times can have a strong effect on the inferred value of α , because these observations are among the most sensitive to the diffusion coefficient. Conversely, errors affecting observations close to the boundaries or at $t = 0$ may have less influence on the parameter estimate, although they can still affect the reconstruction of the temperature field.

Consequently, two datasets generated with the same number of observations and the same nominal noise standard deviation may lead to noticeably different estimates of α . Testing several random realizations is therefore necessary to distinguish systematic behavior from the effect of a particular noise sample.

Generation of synthetic noisy observations For the synthetic experiments, the observation locations (x_i, t_i) are randomly sampled in the space-time domain. Additive Gaussian noise is then introduced according to

$$T_{\text{obs},i} = T_{\text{exact}}(x_i, t_i; \alpha_{\text{theo}}) + \varepsilon_i, \quad (121)$$

with

$$\varepsilon_i \sim \mathcal{N}(0, \sigma_{\text{noise}}^2). \quad (122)$$

The noise standard deviation is defined by

$$\sigma_{\text{noise}} = \eta \text{std}(T_{\text{clean}}), \quad (123)$$

where T_{clean} denotes the set of noise-free temperature values at the observation locations and η is the prescribed relative noise level.

Thus, $\eta = 0.20$ means that the standard deviation of the additive noise is equal to 20% of the standard deviation of the clean dataset. It does not mean that every observation has a pointwise error equal to 20% of its temperature value.

The distinction between the sampling distribution and the noise distribution is important. The observation points may be sampled uniformly or by Latin hypercube sampling, while the additive measurement errors follow a Gaussian distribution. The dataset is therefore described as containing randomly sampled observation points with additive Gaussian noise.

Theoretical, fitted, and PINN-estimated parameters For a finite noisy dataset, the theoretical value α_{theo} used to generate the clean solution does not necessarily minimize the squared error between the exact model and the noisy observations. To quantify this effect independently of the PINN, a reference least-squares estimate can be defined as

$$\alpha_{\text{fit}} = \underset{\alpha > 0}{\operatorname{argmin}} \frac{1}{N_d} \sum_{i=1}^{N_d} [T_{\text{exact}}(x_i, t_i; \alpha) - T_{\text{obs},i}]^2. \quad (124)$$

Because of the finite number of observations and the particular realization of the noise, α_{fit} may differ from α_{theo} . This discrepancy generally becomes more variable as the noise increases and the number of observations decreases.

The least-squares fit therefore provides a useful reference for interpreting the PINN result. Three distinct quantities should be compared:

$$\alpha_{\text{theo}} : \quad \text{parameter used to generate the clean synthetic solution,} \quad (125)$$

$$\alpha_{\text{fit}} : \quad \text{parameter that best fits the finite noisy dataset using the exact model,} \quad (126)$$

$$\alpha_{\text{PINN}} : \quad \text{parameter identified by the inverse PINN.} \quad (127)$$

When the PINN data term is based on the squared L^2 error, it is natural for α_{PINN} to be influenced by the same noisy-data optimum as α_{fit} . The PINN may therefore converge closer to α_{fit} than to α_{theo} .

This behavior does not necessarily indicate a failure of the optimization. It may instead show that the particular noisy dataset statistically favors a parameter different from the value used to generate the clean data. However, the PINN is not mathematically required to reproduce α_{fit} , because its optimization also includes the physics loss, neural-network approximation error, parameterization choices, and optimizer-related effects.

For a single noisy dataset, recovering α_{theo} exactly is generally impossible without additional information about the measurement process or a prior assumption on α . The objective is therefore to obtain an estimate whose uncertainty is consistent with the amount and quality of the available information.

Experimental estimation using repeated measurements For synthetic data, the noise distribution and its standard deviation are known because they are prescribed during data generation. For experimental data, however, the clean temperature field and the probability distribution of the measurement errors are generally unknown.

The most direct method for estimating experimental noise is to perform repeated measurements under nominally identical conditions. If M measurements

$$T_i^{(1)}, \dots, T_i^{(M)} \quad (128)$$

are recorded at the same location x_i and time t_i , their sample mean is

$$\bar{T}_i = \frac{1}{M} \sum_{m=1}^M T_i^{(m)}, \quad (129)$$

and the local measurement variance can be estimated as

$$\hat{\sigma}_i^2 = \frac{1}{M-1} \sum_{m=1}^M \left(T_i^{(m)} - \bar{T}_i \right)^2. \quad (130)$$

This procedure provides a local estimate of the experimental repeatability. If the variance remains approximately constant throughout the dataset, a global noise variance may be obtained by pooling the repeated measurements. If the variance depends on position, time, or temperature amplitude, the measurement noise should instead be treated as heteroscedastic.

Another useful procedure is to record the sensor output while the physical system is maintained in a stationary condition. The fluctuations around the mean then provide an estimate of the background noise generated by the sensor and acquisition system. The characterization should ideally include the entire measurement chain:

- the sensor;

- amplification and signal-conditioning electronics;
- analog-to-digital conversion;
- electromagnetic and environmental interference;
- and any filtering or preprocessing applied to the signal.

Independent calibration measurements can additionally reveal systematic bias, drift, limited temporal response, or nonlinear sensor behavior. These effects cannot always be represented by simple independent Gaussian noise.

Noise estimation from a single experimental dataset Repeated experiments are not always available, particularly when experiments are expensive, destructive, or difficult to reproduce. In this situation, an effective noise level can be estimated from the residuals of a fitted physical model.

Let

$$T_{\text{fit},i} = T_{\text{model}}(x_i, t_i; \hat{\alpha}) \quad (131)$$

denote the temperature predicted after fitting a reference model to the observations. The residuals are defined as

$$r_i = T_{\text{obs},i} - T_{\text{fit},i}. \quad (132)$$

Under the assumptions that the model is sufficiently accurate and that the measurement errors are independent, zero-mean, and homoscedastic, the noise standard deviation may be estimated by

$$\hat{\sigma}_{\text{noise}} = \sqrt{\frac{\sum_{i=1}^{N_d} r_i^2}{N_d - p}}, \quad (133)$$

where p is the number of parameters fitted from the data. In the present analytical benchmark, the exact heat-equation solution can be fitted to obtain α_{fit} , after which the corresponding residuals can be used to estimate the effective noise amplitude.

Since the classical standard deviation is sensitive to anomalous measurements, a robust alternative is the median absolute deviation:

$$\hat{\sigma}_{\text{MAD}} = 1.4826 \text{ median}_i [|r_i - \text{median}_j(r_j)|]. \quad (134)$$

The factor 1.4826 makes the estimator consistent with the standard deviation when the residuals follow a Gaussian distribution. A substantial difference between the classical residual standard deviation and $\hat{\sigma}_{\text{MAD}}$ may indicate the presence of outliers or a non-Gaussian error distribution.

The residuals should not be reduced to a single numerical indicator. They should also be inspected as functions of x , t , and the predicted temperature. Their mean, variance, and correlation structure provide information about the nature of the discrepancies:

- a nonzero residual mean may indicate a systematic measurement or model bias;
- a residual variance that changes with temperature may indicate heteroscedastic noise;
- temporal correlation may reveal sensor dynamics, electronic filtering, or environmental drift;

- spatial patterns may reveal inaccurate boundary conditions or missing physical effects;
- and isolated large residuals may indicate outliers or acquisition failures.

Residual-based noise estimation must nevertheless be interpreted carefully. The residuals may contain not only measurement noise but also model-form error, calibration bias, inaccurate initial or boundary conditions, unresolved physical mechanisms, and numerical or optimization errors. Consequently, the estimated quantity is an effective uncertainty level rather than a pure measurement of the instrumental noise.

Repeated measurements and independent sensor calibration remain preferable whenever they are experimentally feasible.

Signal-to-noise ratio The relevance of a given noise amplitude depends on the amplitude of the useful temperature signal. A signal-to-noise ratio can be defined from the root-mean-square signal as

$$\text{SNR} = \frac{\sqrt{\frac{1}{N_d} \sum_{i=1}^{N_d} T_{\text{clean},i}^2}}{\sigma_{\text{noise}}}. \quad (135)$$

It may also be expressed in decibels as

$$\text{SNR}_{\text{dB}} = 20 \log_{10} (\text{SNR}). \quad (136)$$

For experimental data, the clean signal is unavailable and must be replaced by a fitted or filtered estimate. The resulting SNR should therefore be reported as an estimated quantity.

A global SNR may conceal important local differences. In the rapid-diffusion regime, the SNR can be acceptable at early times but extremely low at later times after the temperature has decayed. Local or time-dependent SNR estimates may consequently be more informative than a single global value.

A high signal amplitude alone does not guarantee strong parameter identifiability. Measurements must simultaneously have an acceptable SNR and a significant sensitivity to α . A useful qualitative indicator of the local information about the parameter is therefore

$$\mathcal{I}_{\alpha,i} \propto \frac{S_{\alpha}(x_i, t_i)^2}{\sigma_i^2}. \quad (137)$$

This expression combines the parameter sensitivity of an observation with its experimental uncertainty. It is closely related to the contribution of that observation to the Fisher information under independent Gaussian errors.

Incorporating the estimated uncertainty into the data loss Once the noise variance has been estimated, it can be incorporated into the training objective. For independent homoscedastic Gaussian noise, the negative-log-likelihood formulation leads, up to additive and multiplicative constants, to

$$\mathcal{L}_{\text{data}} = \frac{1}{N_d} \sum_{i=1}^{N_d} \frac{[T_{\theta}(x_i, t_i) - T_{\text{obs},i}]^2}{\hat{\sigma}_{\text{noise}}^2}. \quad (138)$$

When a conventional unnormalized mean-squared data loss is used, this formulation corresponds to selecting

$$\lambda_{\text{data}} \propto \frac{1}{\hat{\sigma}_{\text{noise}}^2}. \quad (139)$$

The exact numerical value of λ_{data} still depends on the normalization of the input variables, the output temperature, and the physics residual. The inverse-variance relation provides a statistically motivated relative scaling, but does not remove the need to verify the balance between the different loss components.

If the measurement uncertainty differs among observations, the appropriate heteroscedastic data loss is

$$\mathcal{L}_{\text{data}} = \frac{1}{N_d} \sum_{i=1}^{N_d} \frac{[T_{\theta}(x_i, t_i) - T_{\text{obs},i}]^2}{\hat{\sigma}_i^2}. \quad (140)$$

Measurements with greater experimental uncertainty then contribute less to the parameter estimation.

These uncertainty weights should be distinguished from the sensitivity weights introduced in Section ???. The factor $1/\hat{\sigma}_i^2$ represents confidence in the accuracy of an observation, whereas a sensitivity-based factor represents the amount of information that the observation carries about α .

If both effects are included, a combined weighted data loss may be written as

$$\mathcal{L}_{\text{data}}^{(w)} = \frac{\sum_{i=1}^{N_d} \frac{w_i^{(\text{sens})}}{\hat{\sigma}_i^2} [T_{\theta}(x_i, t_i) - T_{\text{obs},i}]^2}{\sum_{i=1}^{N_d} \frac{w_i^{(\text{sens})}}{\hat{\sigma}_i^2}}, \quad (141)$$

where $w_i^{(\text{sens})}$ is a sensitivity-based weight. This formulation gives greater importance to observations that are simultaneously informative about α and experimentally reliable. However, excessive concentration of the weights may reduce the effective number of observations and increase sensitivity to individual noisy points. Weight clipping, a nonzero lower bound, or a square-root transformation may therefore be necessary.

Implications for real inverse problems These results emphasize that the quality of parameter identification depends not only on the PINN architecture or optimization algorithm, but also on the experimental dataset itself. The number of observations, their locations, the measurement uncertainty, the signal-to-noise ratio, and the possible correlation or bias of the errors all affect the recoverability of α .

For a real application, the measurement system should therefore be considered as part of the inverse problem. Relevant information includes the sensor type, calibration procedure, acquisition frequency, temporal response, electronic noise, filtering operations, and expected dependence of the uncertainty on the measured temperature.

Whenever possible, the experimental procedure should include repeated measurements, baseline recordings, and independent calibration. If only one dataset is available, residual analysis can provide an estimate of the effective uncertainty, but cannot completely separate instrumental noise from model discrepancy.

In difficult low-SNR applications, preprocessing may be required to remove identified background contributions, compensate for sensor drift, reject outliers, or account for correlated noise. Such processing must be performed carefully because excessive smoothing can also remove physically meaningful information about the parameter.

Overall, reliable parameter identification requires both an appropriate inverse model and sufficiently informative observations. A small PDE residual and a visually accurate temperature field are not sufficient evidence that the diffusion coefficient has been correctly identified. The inferred parameter must be interpreted relative to the noise realization, the least-squares

reference estimate, the estimated experimental uncertainty, and the sensitivity distribution of the observations.

6.7 Negative Temperature Measurements

An additional difficulty encountered during the generation of synthetic noisy datasets concerns the appearance of negative temperature measurements. The observations were generated by adding independent Gaussian noise to the exact solution:

$$T_{\text{obs},i} = T_{\text{exact}}(x_i, t_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2). \quad (142)$$

For the initial and boundary conditions considered in this work, the exact solution is given by

$$T_{\text{exact}}(x, t) = \sin\left(\frac{\pi x}{L}\right) \exp\left(-\alpha \frac{\pi^2}{L^2} t\right), \quad (143)$$

and therefore satisfies

$$T_{\text{exact}}(x, t) > 0, \quad x \in (0, L), \quad t \geq 0. \quad (144)$$

However, Gaussian noise has unbounded support. Consequently, a sufficiently large negative noise realization can produce

$$T_{\text{obs},i} < 0 \quad (145)$$

even though the underlying physical solution remains positive. The probability of obtaining a negative observation at a given point is

$$\mathbb{P}(T_{\text{obs},i} < 0) = \Phi\left(-\frac{T_{\text{exact}}(x_i, t_i)}{\sigma}\right), \quad (146)$$

where Φ denotes the cumulative distribution function of the standard normal distribution. Negative observations therefore become increasingly likely when the local signal-to-noise ratio

$$\text{SNR}_i = \frac{T_{\text{exact}}(x_i, t_i)}{\sigma} \quad (147)$$

becomes small.

This situation occurs primarily at late times, close to the spatial boundaries, or for large values of α . In all three cases, the exact temperature approaches zero and the noise amplitude may become comparable to, or larger than, the physical signal. Negative observations are thus not necessarily numerical errors; they are a natural consequence of applying additive Gaussian noise to a positive but decaying signal.

Several preprocessing strategies were investigated.

Keeping the observations unchanged. The first approach consists of retaining all noisy observations, including the negative values. This preserves the assumed zero-mean Gaussian noise distribution and avoids introducing an explicit preprocessing bias. The corresponding data loss is evaluated directly as

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} [T_{\theta}(x_i, t_i) - T_{\text{obs},i}]^2. \quad (148)$$

Although statistically consistent with the adopted noise model, this approach forces the PINN to fit observations that are incompatible with the expected positivity of the temperature field. This can be particularly problematic when many low-amplitude measurements are

negative, since they may pull the predicted solution below zero or distort the inferred decay rate.

Clipping negative observations. A second possibility is to replace every negative measurement by zero:

$$T_{\text{obs},i}^{\text{clip}} = \max(T_{\text{obs},i}, 0). \quad (149)$$

This guarantees non-negative training data and is straightforward to implement. However, clipping changes the statistical properties of the measurement errors. All negative values are concentrated at zero, producing an artificial accumulation of observations at the physical lower bound. In regions where the exact temperature is weak, the resulting errors are no longer Gaussian or zero-mean.

In particular, clipping introduces an upward bias because

$$\mathbb{E}[T_{\text{obs}}^{\text{clip}}] \neq T_{\text{exact}}. \quad (150)$$

This bias may cause the PINN to predict a temperature that decays too slowly. Since the decay rate is controlled by α , the estimated diffusivity may also become biased.

Transforming negative observations. Alternative transformations can also be considered, such as taking the absolute value,

$$T_{\text{obs},i}^{\text{abs}} = |T_{\text{obs},i}|, \quad (151)$$

or applying a smooth positivity-preserving transformation. These approaches retain the number of observations but alter their physical and statistical interpretation. For example, the absolute-value transformation reflects negative noise realizations into the positive domain and therefore systematically increases the mean temperature in low-signal regions.

Such transformations may improve numerical stability, but they no longer correspond to the original additive Gaussian observation model. They must therefore be interpreted as preprocessing heuristics rather than statistically neutral corrections.

Removing negative observations. The final strategy consists of discarding observations that violate the positivity condition:

$$\mathcal{D}_{\text{valid}} = \{(x_i, t_i, T_{\text{obs},i}) \mid T_{\text{obs},i} \geq 0\}. \quad (152)$$

The effective number of data points then becomes

$$N_{\text{eff}} = \sum_{i=1}^{N_{\text{data}}} \mathbb{I}(T_{\text{obs},i} \geq 0), \quad (153)$$

where $\mathbb{I}$ is the indicator function. The fraction of retained observations can be quantified as

$$r_{\text{valid}} = \frac{N_{\text{eff}}}{N_{\text{data}}}. \quad (154)$$

This approach prevents the network from fitting physically inconsistent temperature values. Nevertheless, it reduces the effective dataset size and preferentially removes measurements from low-temperature regions. The retained dataset is therefore no longer an unbiased sample of the original observation distribution. This selection effect can again favor larger observed temperatures and potentially underestimate the true rate of decay.

In the present study, negative measurements were removed before training, and the resulting effective dataset size was systematically recorded. This choice was motivated by the physical

positivity of the temperature field and by the instability observed when negative measurements dominated low-signal regions. However, the retained fraction must be considered when comparing experiments: two datasets generated with the same nominal value of N_{data} may contain substantially different numbers of usable observations.

The frequency of negative measurements also provides useful information about the difficulty of the inverse problem. A large rejected fraction indicates that a significant part of the dataset lies below the noise floor. In this regime, preprocessing alone cannot recover the information lost through weak signal amplitude. The estimation of α becomes intrinsically ill-conditioned because the observations carry little information about the diffusion dynamics.

These experiments demonstrate that the treatment of negative measurements is not merely a numerical implementation detail. Keeping, clipping, transforming, or removing them modifies either the statistical distribution of the data, the physical constraints imposed on the model, or the effective amount of information available during training. Consequently, the selected preprocessing strategy and the effective number of retained observations should always be reported when evaluating inverse PINN performance.

6.8 Adaptive Weighting Strategies

A major part of this work focused on balancing the contributions of the physics and data losses.

The total loss function is written as:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{data}}\mathcal{L}_{\text{data}} + \lambda_{\text{phys}}\mathcal{L}_{\text{phys}} \quad (155)$$

Choosing appropriate weighting coefficients is difficult because:

- noisy datasets require stronger physical regularization,
- while overly strong physics constraints may suppress informative data signals.

Several adaptive weighting strategies were investigated.

These include:

- noise-dependent weighting,
- signal-amplitude weighting,
- and sensitivity-based weighting using:

$$\left| \frac{\partial T}{\partial \alpha} \right| \quad (156)$$

The objective was to emphasize spatial and temporal regions containing the most information regarding α .

These strategies significantly improved parameter reconstruction robustness in several noisy configurations.

Note the presence of lambda parameters in the definition of the loss function. This is to regulate the weight associated with each term of the loss function. For instance, if we feel data that the dataset is noisy it might be preferable to give more weight to the physics to get accurate results in the domain even if the data loss will increase. Conversely, if we know with high confidence that the dataset is good it is important to give more weight to the data loss such that the training will minimize the data loss in priority.

In general context, or if do not know, it is best to get equilibrate data loss term and to adapt the lambdas accordingly. However, in general depending on the region of the spatial domain or advancement in the training, equilibration cannot be achieved with a single scalar and the

values of these lambdas must change during the training. Several adaptive weithing strategies were proposed in the literature namely ???

Note that adapative weigting strategies is not the only way to equilibrate the terms for the physics loss. As shown in section, normalizations of can plays a role too.

6.9 Galerkin-Inspired Constraints

In addition to standard residual minimization, Galerkin-inspired constraints were introduced in order to improve parameter identifiability.

Rather than minimizing only pointwise PDE residuals, the idea consists of projecting the residual onto physically informative modes related to α sensitivity.

A supplementary loss term was therefore introduced:

$$\mathcal{L}_{Gal} = (\langle R, S_\alpha \rangle)^2 \quad (157)$$

where:

- R denotes the PDE residual,
- and S_α represents an α -sensitive weighting function.

The purpose of this strategy is to force the optimization process to focus more directly on parameter-related inconsistencies rather than solely minimizing local residuals.

These experiments suggest that classical pointwise residual minimization may not always be sufficient for robust inverse parameter identification.

6.10 Dynamic MSE Thresholds and Robustness Studies

Another important aspect investigated in this work concerns the evaluation criteria used to determine whether a PINN solution is acceptable.

Fixed Mean Squared Error (MSE) thresholds can become problematic because the acceptable error level naturally depends on:

- the noise amplitude,
- the signal magnitude,
- and the difficulty of the inverse problem.

For this reason, dynamic thresholds depending on the noise level were explored.

For example:

$$\text{MSE}_{data} \sim \sigma^2 \quad (158)$$

where σ denotes the estimated noise amplitude.

This allows the evaluation criteria to remain physically meaningful across different noise regimes.

Extensive robustness studies were performed by varying:

- the noise level,
- the dataset size,
- the initial guess for α ,
- network architecture parameters,
- and loss weighting coefficients.

These studies revealed that:

- apparently small implementation choices can strongly influence inverse PINN behavior,
- parameter identifiability may remain difficult even with accurate solution reconstruction,
- and careful analysis of optimization dynamics is essential.

6.11 Limitations and Perspectives

Although PINNs provide a promising framework for inverse problems, this work also highlights several important limitations:

- optimization stiffness,
- sensitivity to hyperparameters,
- parameter identifiability difficulties,
- and high computational cost compared to standard forward solvers.

For relatively simple forward PDE simulations, classical numerical methods remain significantly faster and more reliable.

However, PINNs become increasingly attractive when:

- sparse experimental data must be integrated,
- unknown physical parameters must be estimated,
- geometries become complex,
- or multiple coupled physical systems are involved.

The detailed study of this benchmark inverse problem therefore provides a valuable foundation before extending the methodology toward more realistic scientific machine learning applications involving nonlinear PDEs, multiphysics systems, and experimental data assimilation.